



移动互联网、人工智能、大数据背景下

高级信息系统

Advanced Information Systems



2022秋高级信息系统

中国人民大学信息学院

蒋洪迅, 15501010106

jianghx@ruc.edu.cn



Part 2

2023/1/7



主要内容

- 一、信息系统开发中的问题与对策
- 二、信息系统工程
- 三、过程
- 四、过程模型
- 五、信息系统工程方法



一、信息系统开发中的问题

上世纪**60**年代末的软件危机引出软件工程

软件危机是指在计算机软件的开发和维护过程中遇到的一系列严重问题。

- ❑ 软件开发的成本和进度难以准确估计，导致延迟交付甚至取消项目
- ❑ 软件存在着错误多、性能低、不可靠、不安全等质量问题
- ❑ 软件维护极其困难，而且很难适应不断变化的用户需求和使用环境



问题和表现

- ❑ 软件开发无计划性
- ❑ 软件需求不充分
- ❑ 开发过程无规范
- ❑ 软件产品无评测手段
- ❑ 软件产品缺少适当文档资料
- ❑ 软件生产率低



背后的原因

- ❑ 开发遗留的问题给维护带来困难
- ❑ 规模增大带来的复杂性增大（非线性）
- ❑ 开发工作中无可自始至终遵循的工作方法
- ❑ 对需求的了解不充分
- ❑



真实的案例，残酷的现实



客户如此描述需求



项目经理如此理解



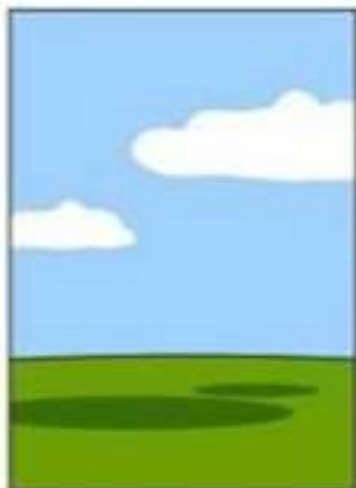
分析员如此设计



程序员如此编码



商业顾问如此诠释



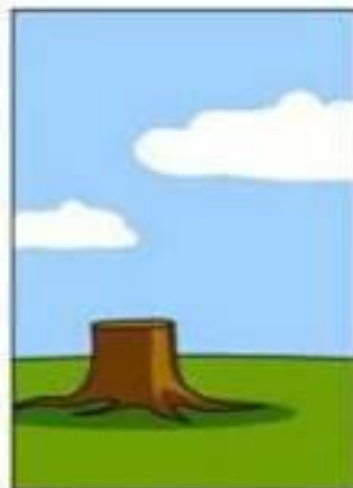
项目文档如此编写



安装程序如此“简洁”



客户投资如此巨大



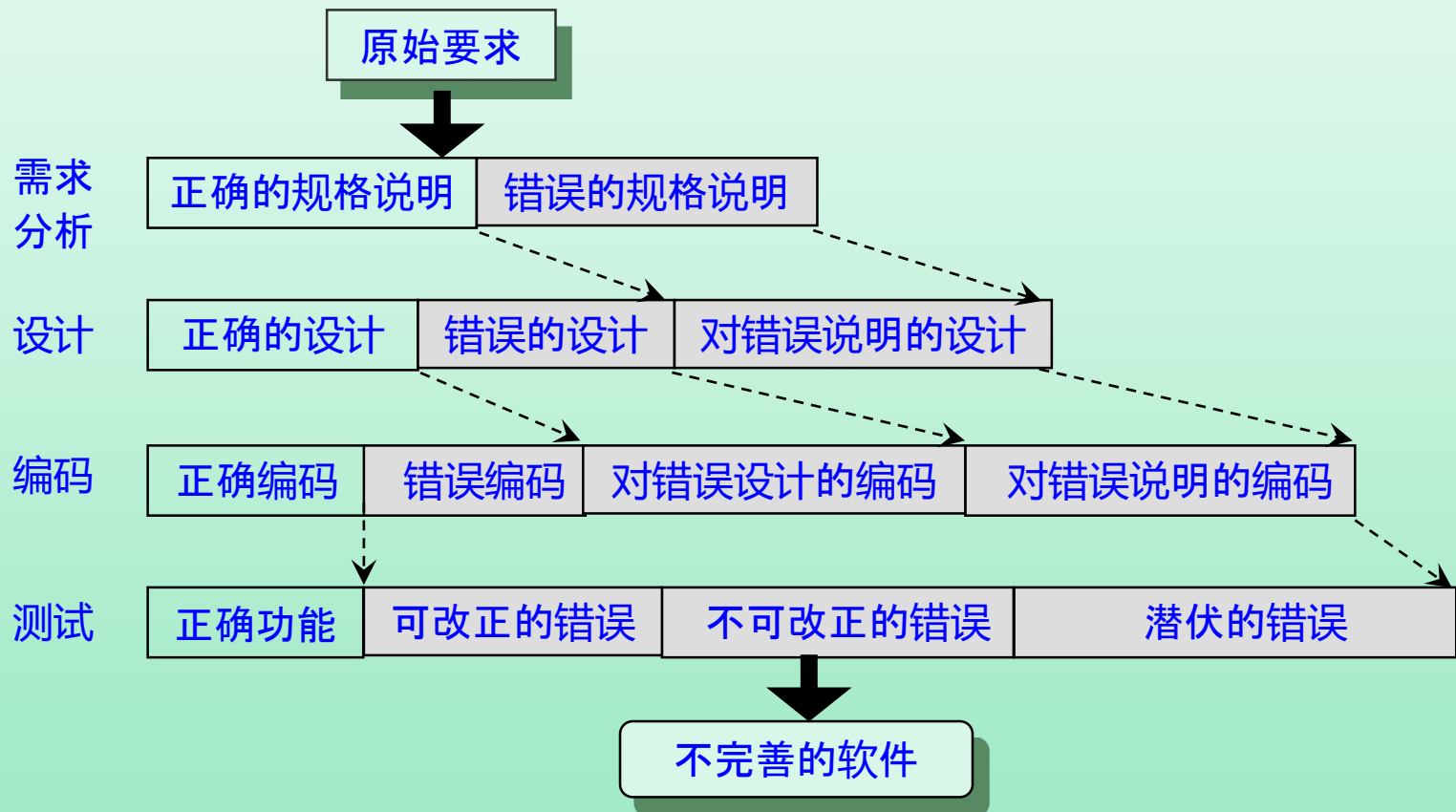
技术支持如此肤浅



解密：
实际需求—原来如此



软件错误的积累与放大效应





二、信息系统工程 (Information System Engineering)

工程是将理论和所学的知识应用于实践的科学，以便经济有效地解决实际问题。

工程的含义（职能：研究、开发、设计、施工、生产）

- 规模上的差异
 - 花园小道 vs. 汽车高速公路
 - 树上小屋 vs. 摩天大楼
 - 加法程序 vs. 医院档案系统



二、信息系统工程 (Information System Engineering)

工程的含义（职能：研究、开发、设计、施工、生产）

- 手工：小规模的设计与建造
 - 简单问题与单一目标
 - 个人控制与个人技能
- 工程 (*Engineering*)：大规模的设计与建造
 - 复杂问题与目标分解
 - 多人参与，需要考虑运营、管理、成本、质量控制、安全等
- 信息系统工程与软件工程



软件工程基本原则

- 较低的开发成本
- 达到要求的功能
- 取得较好的性能
- 易于移植
- 较低的维护费用
- 能按时完成开发工作，及时交付使用



软件工程原理

- 用分阶段的生命周期计划严格管理
- 坚持进行阶段评审
- 实行严格的产品控制
- 采用现代程序设计技术
- 结果应能清楚地审查
- 开发人员应少而精
- 承认不断改进软件工程实践的必要性

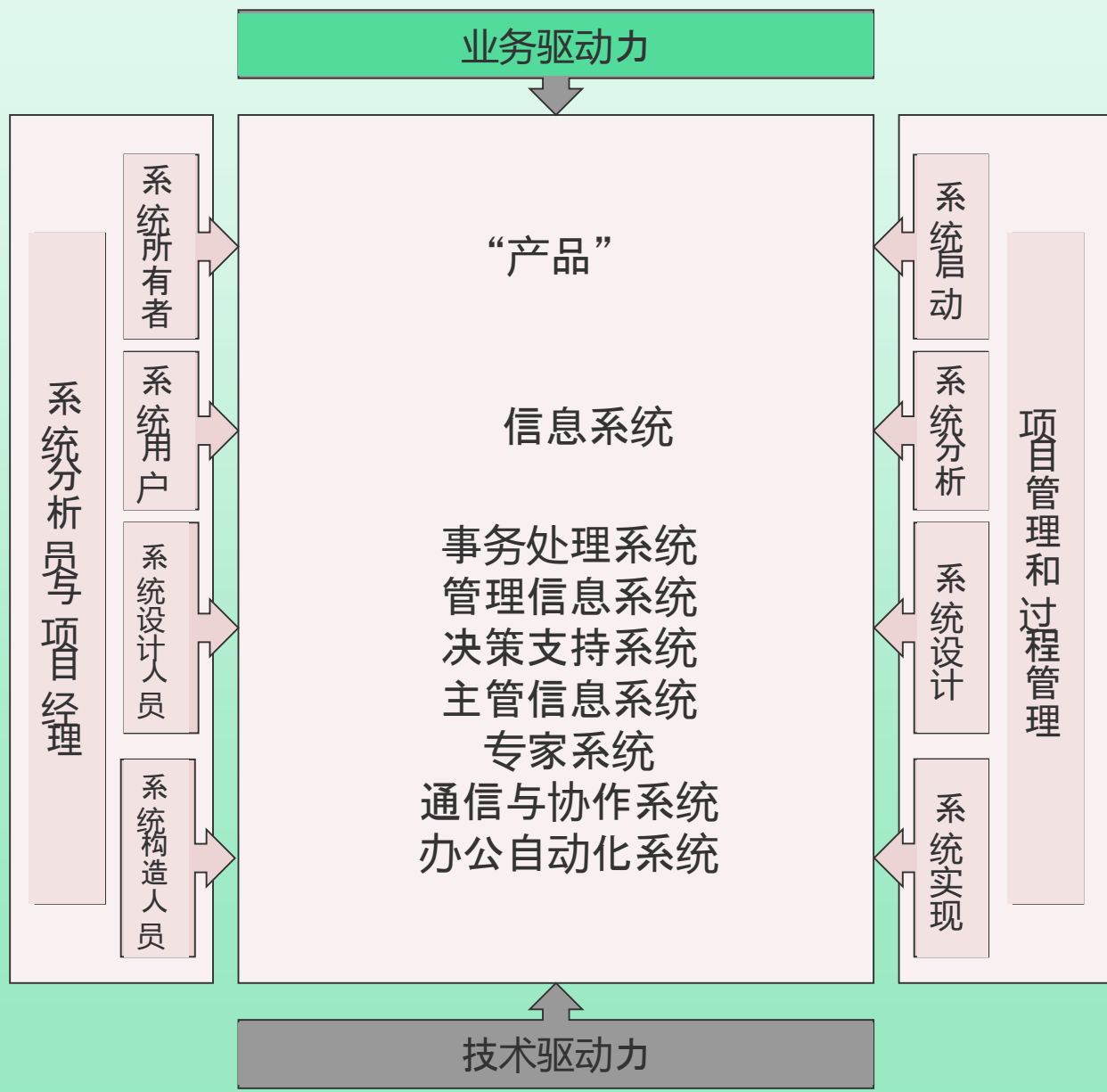


软件工程四项基本活动

- 软件规格说明
- 软件开发
- 软件确认
- 软件演进



信息系统分析与设计角度





信息系统分析与设计的四个角度

■ 参与者角度-系统分析员和项目经理

- 所有权人：成本和效益
- 用户：功能、可用性
- 设计人员：技术方案
- 开发人员：构造和部署
- 系统分析员：业务需求与技术的沟通者
- 项目经理：项目负责人



系统所有者

- 是信息系统的**发起人和主要倡导者**，他们通常负责投资项目以进行开发、运行和维护信息系统。
 - 大中型系统 —— 中高层经理
 - 系统所有者
- 系统所有者往往对结果感兴趣 —— 系统成本是多少？这个系统给企业带来多少价值，或者这个系统将给企业带来多大效益？





系统用户

- 是那些在通常意义上使用信息系统或者受到信息系统影响的人：收集、验证、录入、响应、存储、交换数据和信息。常用的同义词是**客户**。
- 同系统所有者不同，系统用户不关心系统的成本和收益。相反，他们关心系统提供的功能，系统是否易学易用。





内部系统用户

- 办事员和服务人员：通常由他们处理大部分日常事务；
- 技术人员和专业人员：主要有业务专家和行业专家构成，他们进行高技术的和专业化工作；
- 主管、中层经理和高层经理：他们都是决策制定者，往往关心日常的管理问题。



外部系统用户

- 顾客：是指任何购买我们的产品和服务的企业或个人；
- 供应商：向其购买补给品和原材料的任何单位都成为供应商；
- 合作伙伴：从其购买服务或者与之合作的公司称之为合作伙伴；
- 雇员：指那些在外出差或者在家工作的雇员；
- 外部系统用户越来越多的指远程用户和移动用户，他们通过笔记本电脑、掌上电脑和智能手机连接到信息系统中。



系统设计人员

- 将系统用户的业务需求和约束条件转换为技术方案。他们设计满足系统用户需求的计算机文件、数据库、输入、输出、屏幕界面等等。**他们对信息技术的选择和使用所选技术设计系统感兴趣。**
- 包括：数据库管理员、网络架构师、Web架构师、图形艺术家、安全专家、技术专家。





系统架构人员

- **根据系统设计人员的设计说明构造信息系统架构。**在许多情况下，一个构件的系统设计人员和系统构架人员是同一组人员。但在大型组织或大型信息系统中，通常由不同的人员来担任
- 包括：应用程序员、系统程序员、数据库程序员、网络管理员、安全管理员、Web站点管理员、软件集成员。





系统分析员

- 系统所有者、用户、设计人员和构造人员对要构造和使用的信息系统经常存在不同的视点，因此他们之间沟通可能会存在交流障碍，系统分析员就是要沟通这个障碍。
- 系统分析员研究组织中存在的问题和需求，决定人、数据、过程、通信和信息技术如何最好地对企业业务进行改进。
- （在学习这门课的时候，在座各位可能要扮演一个系统分析员的角色）

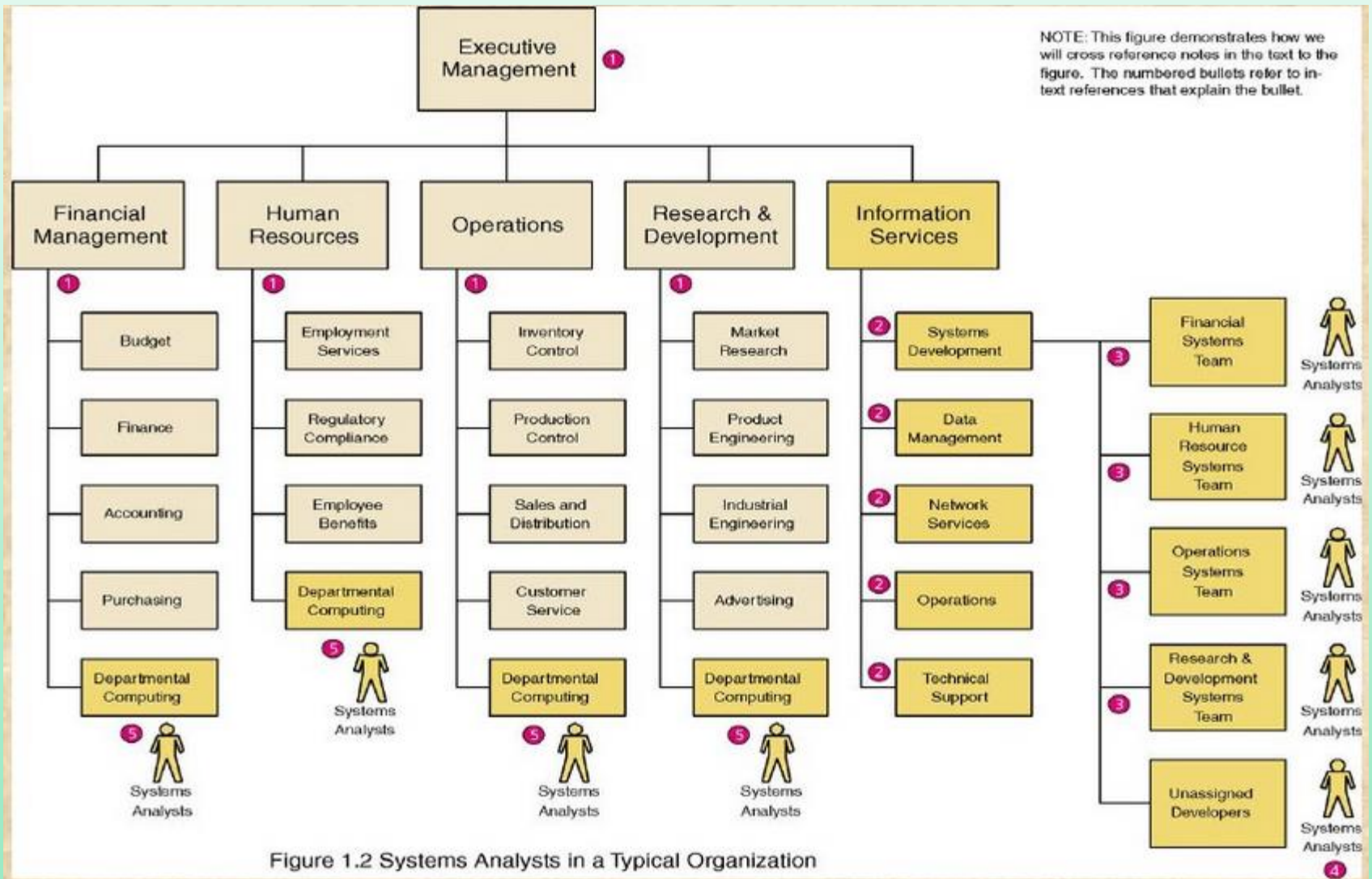


系统分析员的角色

- 系统分析员基本上是一个解决问题的人。
- 问题：
 - 真正的问题，对这些问题需要采取正确的行动；
 - 尽管还没有人抱怨，但可以改进现状的机会；
 - 无论是否有人抱怨，都要改变现状的指示。
- 系统分析员为高层管理人员作出重要决策提供了可预见性和机会，以及可能影响整个组织的创造力。

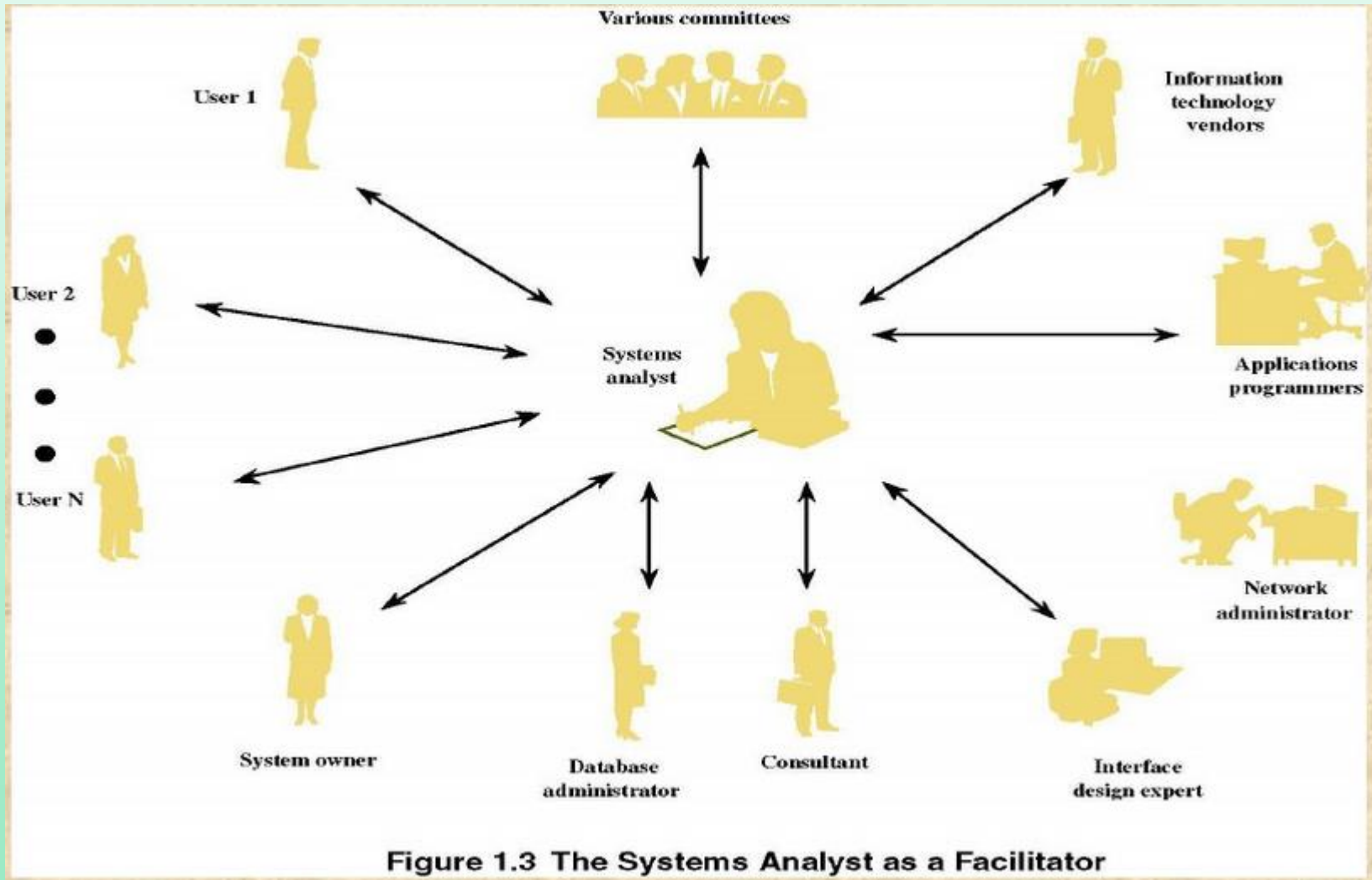


系统分析员在哪儿工作





系统分析员的职业前景





系统分析员所需的技能

- 有效的信息技术知识；
- 计算机编程经验和专长；
- 一般商业知识；
- 通用的解决问题的技能；
- 良好的与人沟通的能力；
- 良好的处理人际关系的能力；
- 灵活性和适应能力；
- 人格与道德规范。



外部服务提供者

- 销售专业知识和经验给其他企业，帮助那些企业购买、开发、集成信息系统；
 - 可以是系统分析员、系统设计人员、构造人员；
 - 隶属咨询机构或者服务机构。
-
- 举例：技术工程师、销售工程师、系统顾问、签约程序员、系统集成人员。



项目经理

- 经验丰富的从业人员，根据进度安排、预算、发布产品、客户满意度、技术标准、系统质量，计划、监视和控制项目；
- 项目管理是一个专业角色，需要专门技能和经验；
- 大多数项目经理由系统分析员担任，有些事从系统所有者中挑选。



信息系统分析与设计的四个角度

- 企业驱动力-产品
 - 经济全球化
 - 电子商务和电子业务
 - 安全和隐私
 - 协作与合伙经营
 - 知识产权保护
 - 持续改进
 - 全面质量管理
 - 业务过程重组



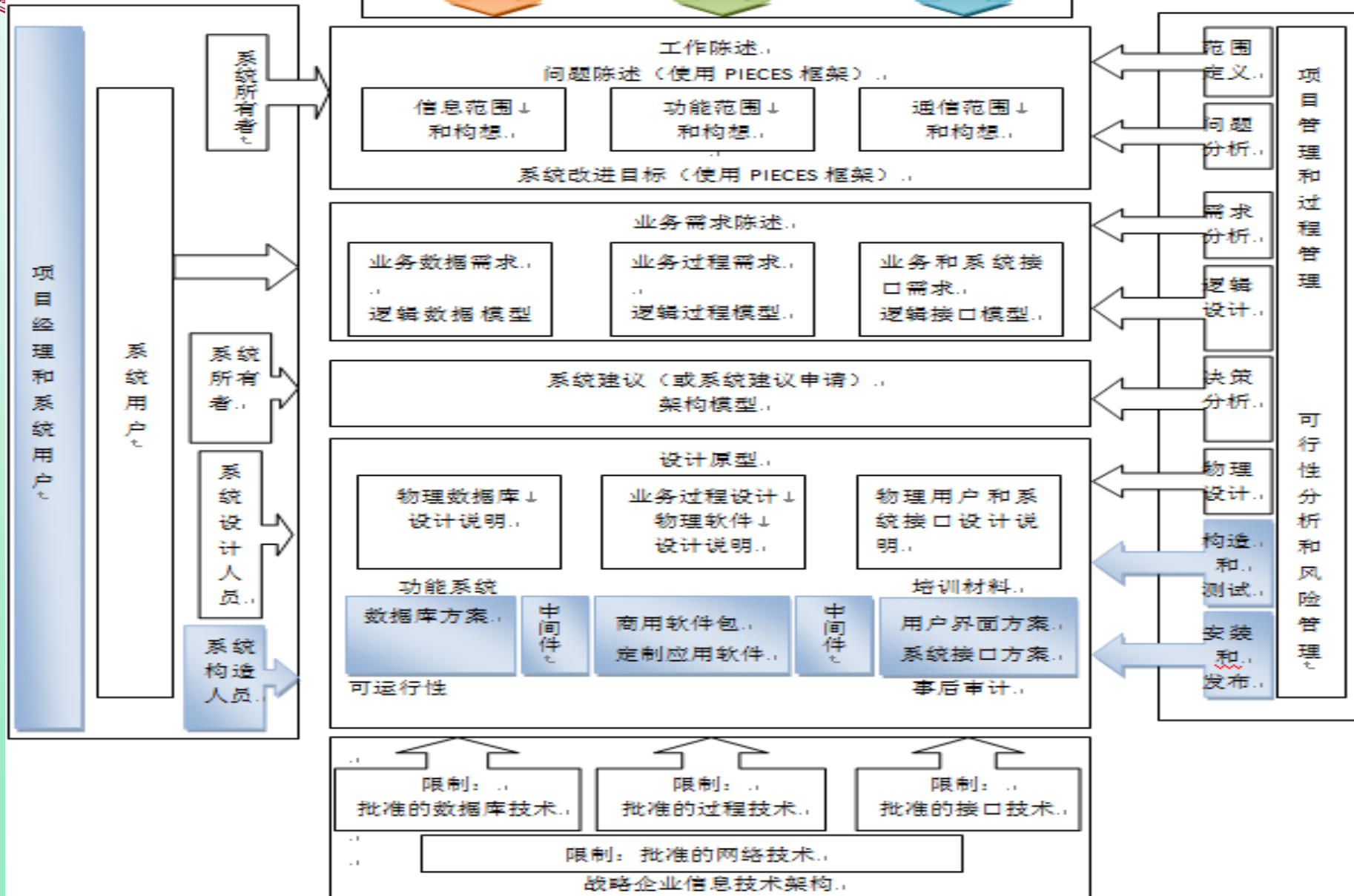
信息系统分析与设计的四个角度

- 技术驱动力
 - 网络和互联网
 - 移动无线技术
 - 对象技术
 - 协作技术
 - 企业应用软件



信息系统分析与设计的四个角度

- 过程与管理-项目管理和过程管理
 - 系统启动
 - 系统分析
 - 系统设计
 - 系统实现
 - 系统支持和持续改进





软件工程三大要素

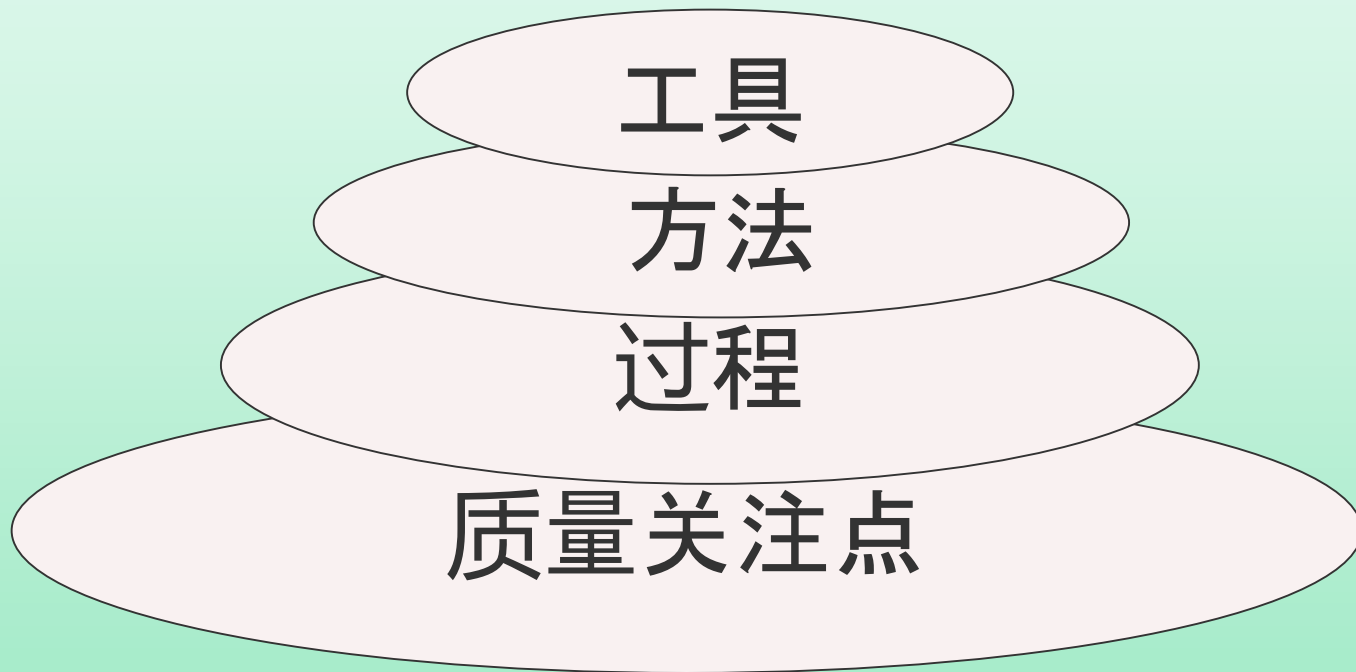
过程、方法和工具。

- 过程
 - 支持软件生命周期的所有活动
- 方法
 - 为软件开发过程提供“如何做”的技术
- 工具
 - 为软件开发方法提供自动的或半自动的软件支撑环境





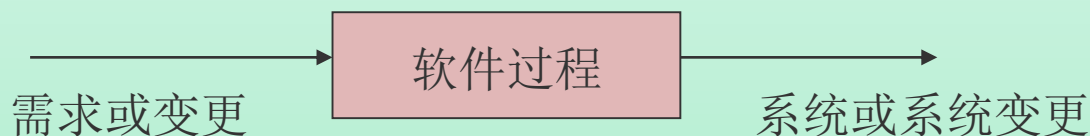
三、过程





软件过程

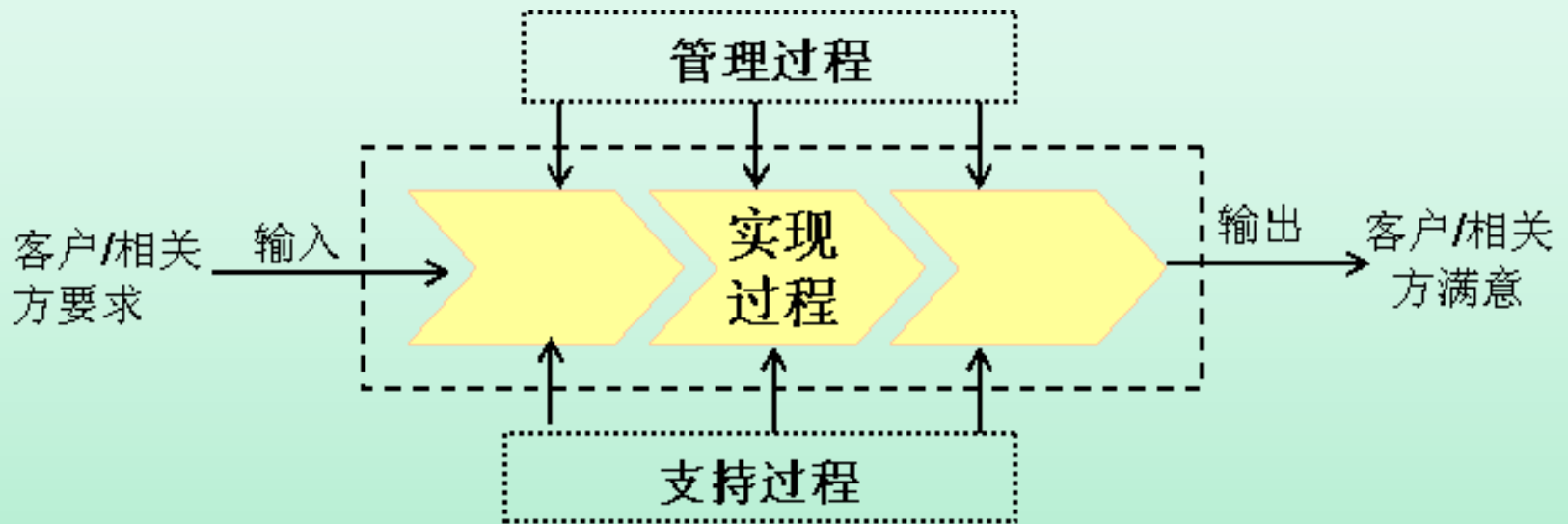
- 为软件开发和演变提供指南。它奠定了以后各代产品的基础；降低风险提高质量；从经验中获取最佳实践并记录下来。



- 为开发提供指南
- 降低风险
- 提供最佳实践
- 共识和文化



实现、管理和支持过程间关系



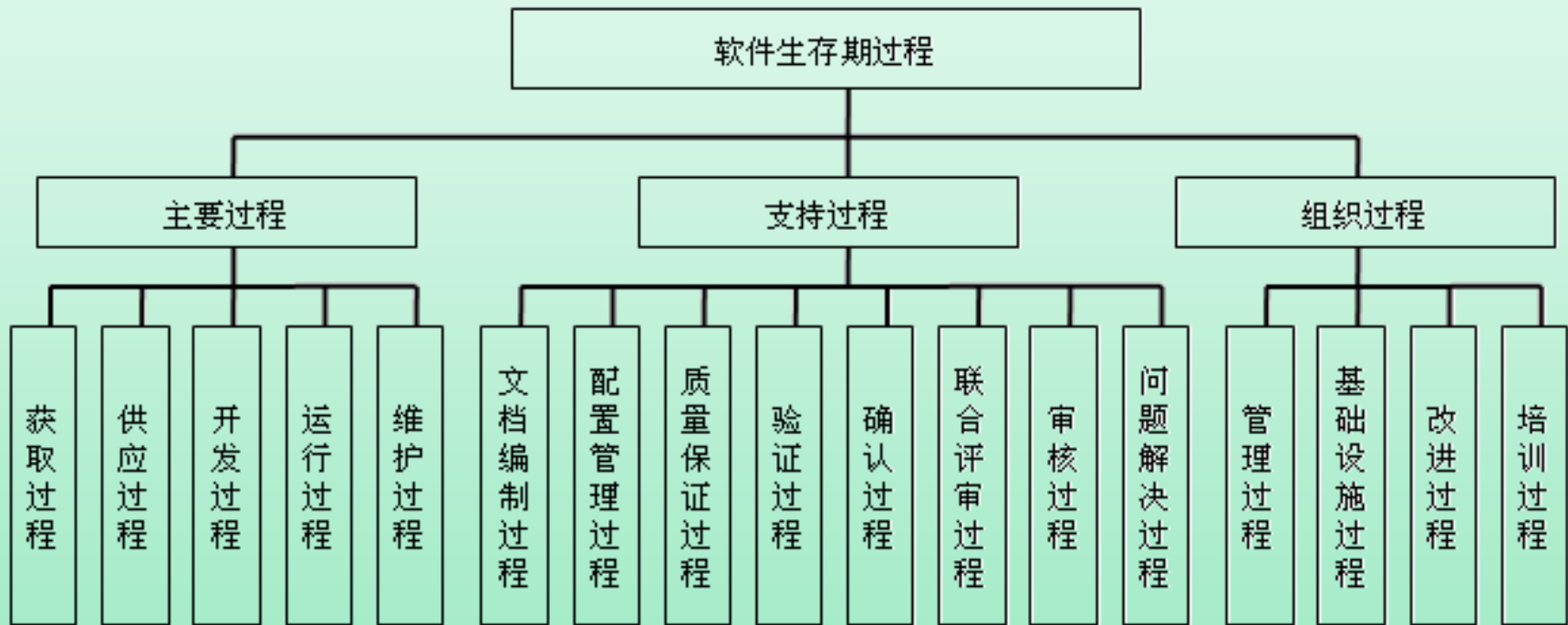
■ 实现过程

■ 管理过程

■ 支持过程

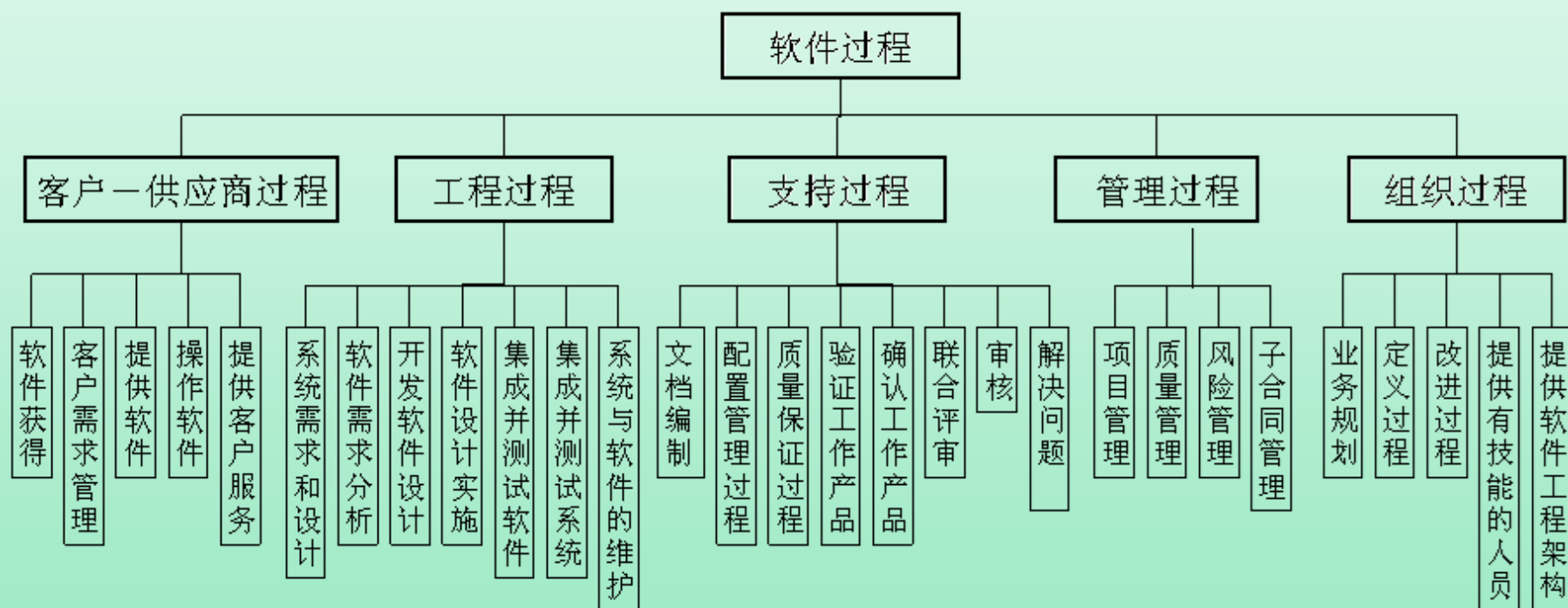


IEC12207软件生存周期过程





ISO/IEC15504软件生存周期过程





CMM

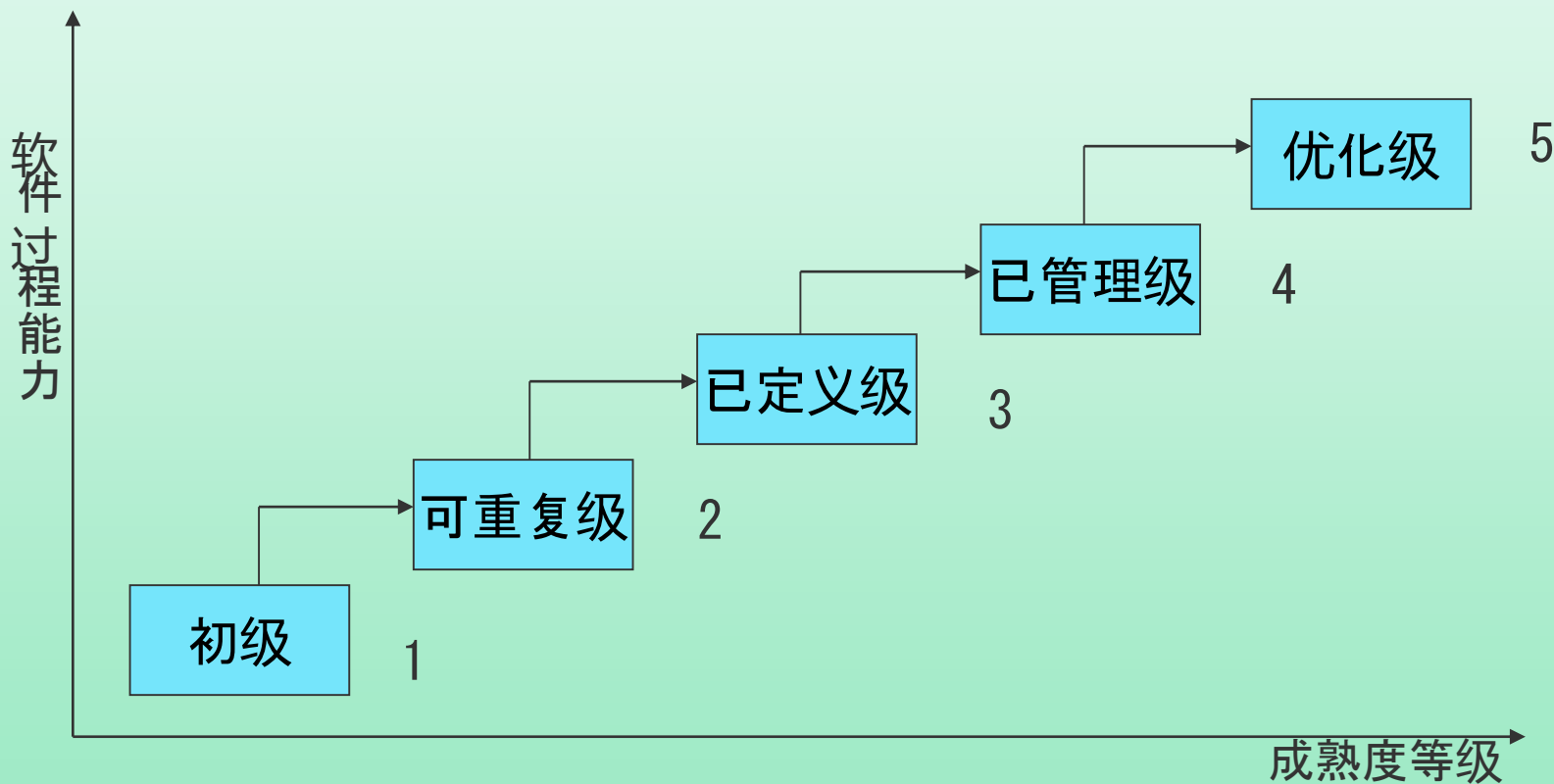
- Capability Maturity Model for Software,英文缩写为SW-CMM,简称CMM
- 软件过程能力成熟度是指一个特定过程被明确地定义、管理、测量、控制并且是有效的程度。成熟度意味着能力上的增长能力，并表明一个组织软件过程的丰富性和在项目中运用它时的一致性

软件工程与CMM标准之父、美国科学院院士、美国卡内基·梅隆大学软件工程研究所(CMU SEI)执行所长安吉尔.G. 乔丹





CMM的五级模型



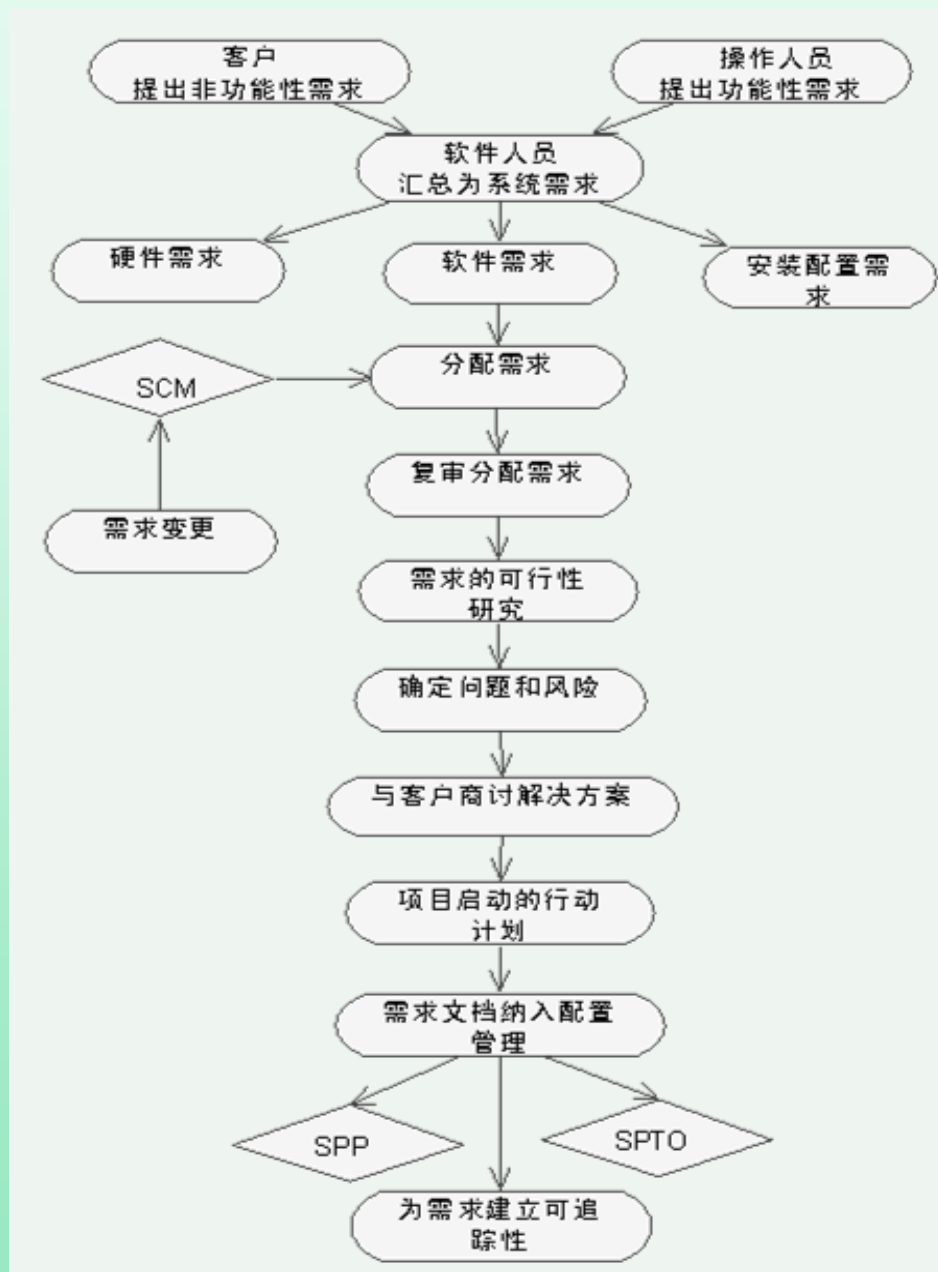


关键过程域(KPA)

等级 \ 过程分类	管理方面	组织方面	工程方面
优化级		技术改革管理 过程变更管理	缺陷防范
可管理级	定量过程管理		软件质量管理
已定义级	集成软件管理 组间协调	组织过程焦点 组织过程定义 培训程序	软件产品工程 同级评审
可重复级	需求管理 软件项目计划 软件项目跟踪与监控 软件转包合同管理 软件质量保证 软件配置管理		
初始级	无序过程		



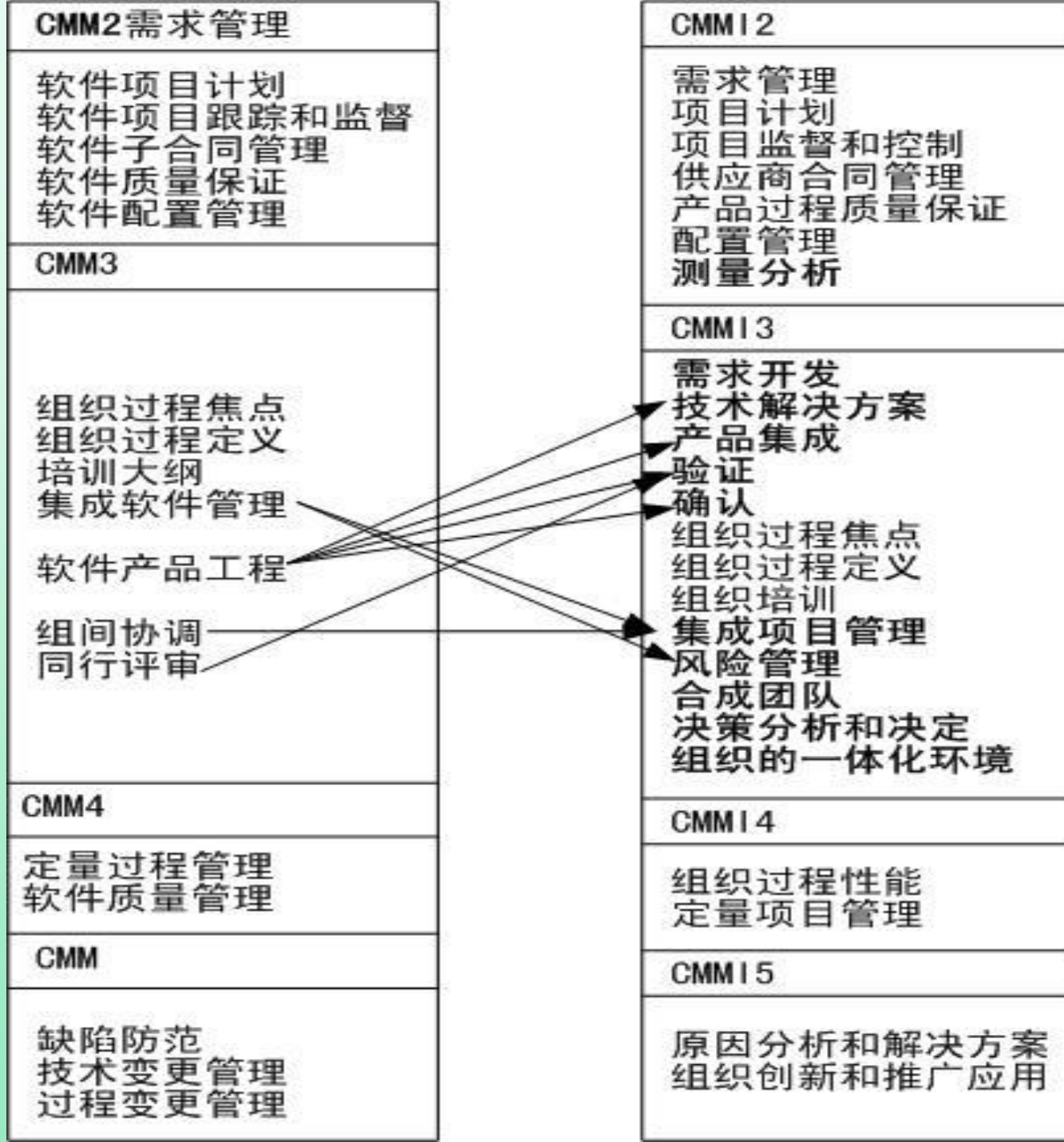
需求管理





CMMI

- CMMI (*Capability Maturity Model integration*) 是为了解决现有不同CMM模型的重复性、复杂性，由美国卡耐基梅隆大学软件工程研究院（SEI）开发的能力成熟度模型集成，它将软件CMM2.0版草案C(SW-CMM)、EIA过渡标准731（系统工程CMM）及IPD-CMM集成为一体，解决了多模型之间的重叠问题。同时，还与ISO15504相兼容。
- SEI于2001年12月发布了CMMI 1.1版本，同时发表了针对CMMI 的一套评估体系 SCAMPI SM V1.1 。
- CMMI模型包括了一系列的过程域，这些过程域构成了集成能力模型的核心，提供了软件工程、系统工程、集成产品及过程开发的过程改进指南。CMMI模型包括阶段式成熟度能力和持续式过程能力两种表示方法。CMMI模型可以根据企业使命和商业目标进行裁剪以符合企业自身的需要。





COBIT

- COBIT (Control Objectives for Information and related Technology) 是国际上通用的信息系统审计的标准，由信息系统审计与控制协会 (ISACA) 在1996年公布。这是一个在国际上公认的、权威的安全与信息技术管理和控制的标准，已经更新至COBIT2019版。它在商业风险、控制需要和技术问题之间架起了一座桥梁，以满足管理的多方面需要。该标准体系已在世界一百多个国家的重要组织与企业中运用，指导这些组织有效利用信息资源，有效管理与信息相关的风险。



COBIT信息技术的控制目标

- ❑ **有效性 (Effectiveness)** ——是指信息与商业过程相关，并以及时、准确、一致和可行的方式传送。
- ❑ **高效性 (Efficiency)** ——关于如何最佳（最高产和最经济）利用资源来提供信息。
- ❑ **机密性 (Confidentiality)** ——涉及对敏感信息的保护，以防止未经授权的披露
- ❑ **完整性 (Integrity)** ——涉及信息的精确性和完全性，以及与商业评价和期望相一致
- ❑ **可用性 (Availability)** ——指在商业处理需求中，信息是可用的。还指对必要的资源和相关性能的维护。
- ❑ **符合性 (Compliance)** ——遵守商业运作过程中必须遵守的法律、法规和契约条款，如外部强制商业标准。
- ❑ **信息可靠性 (Reliability of Information)** ——为管理者的日常经营管理以及履行财务报告责任提供适当的信息。



四、工程过程模型

- 软件过程模型
 - 软件过程模型是对实际过程的抽象描述
 - 包括软件过程的活动、软件产品以及参与人员的不同角色
- 模型划分
 - 接受式
 - 直线式
 - 迭代式
 - 实验式
 - 规划式

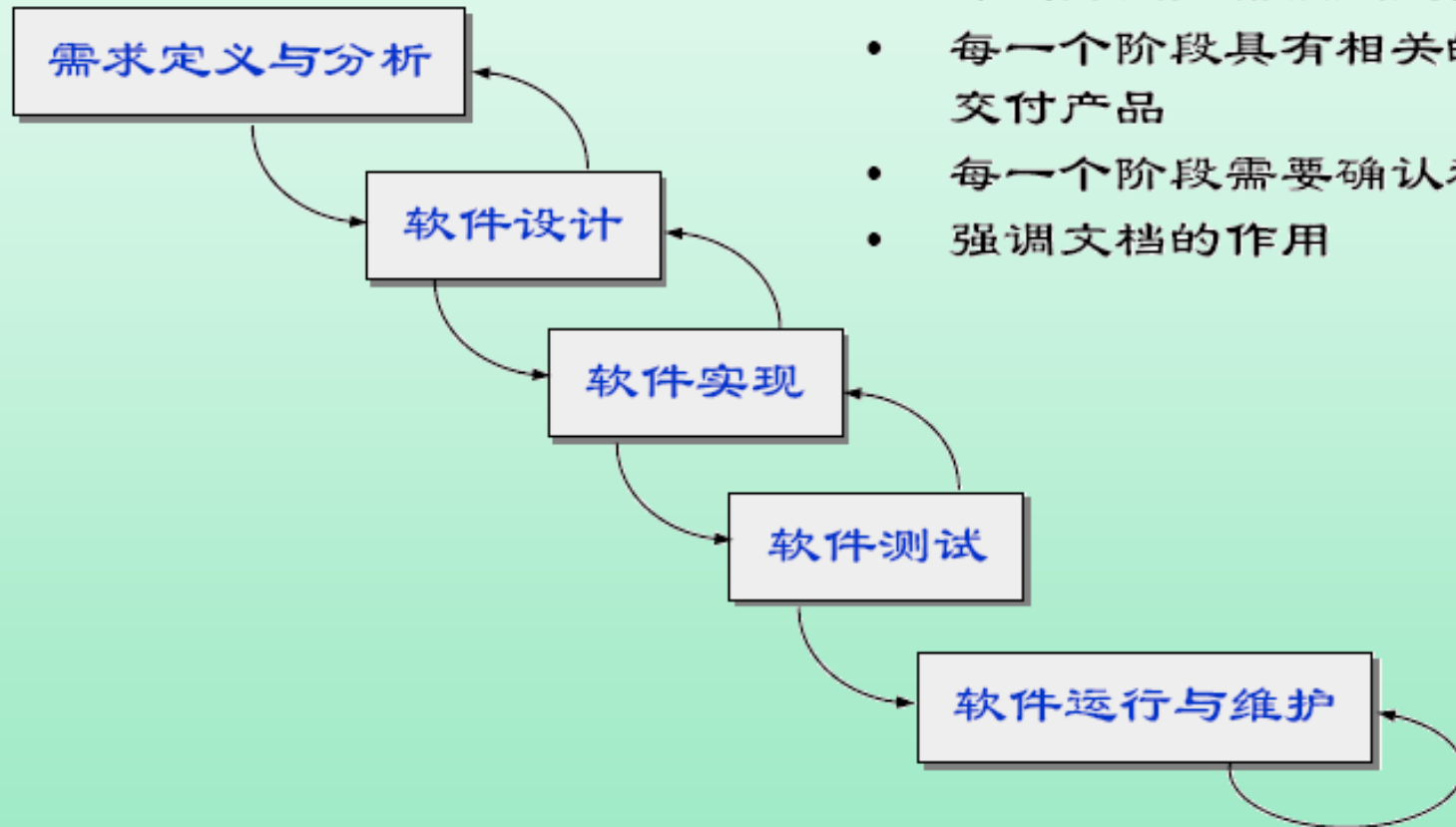


过程模型

- 常见的软件过程模型
 - 瀑布模型
 - 快速原型模型
 - 增量模型
 - 螺旋模型
 - 形式化方法模型
 - 基于组件的开发模型
 - 其他



软件生存期模型——瀑布模型



- 开发阶段严格按照线性方式进行
- 每一个阶段具有相关的里程碑和交付产品
- 每一个阶段需要确认和验证
- 强调文档的作用



信息系统的生命周期

- 任何事物都有产生、发展、成熟、消亡(更新)的过程，信息系统也不例外。信息系统在使用过程中随着其生存环境的变化，要不断维护、修改，当它不再适应的时候就要被淘汰，就要由新系统代替老系统，这种周期循环称为信息系统的生命周期。
- 信息系统的生命周期可以分为系统规划、系统分析、系统设计、系统实施、系统运行和维护五个阶段。



瀑布模型

□ 适用

- 在开发的早期阶段软件需求被完整确定

□ 挑战

- 实际的项目开发很少是线性的过程，客户很难明确地描述软件需求

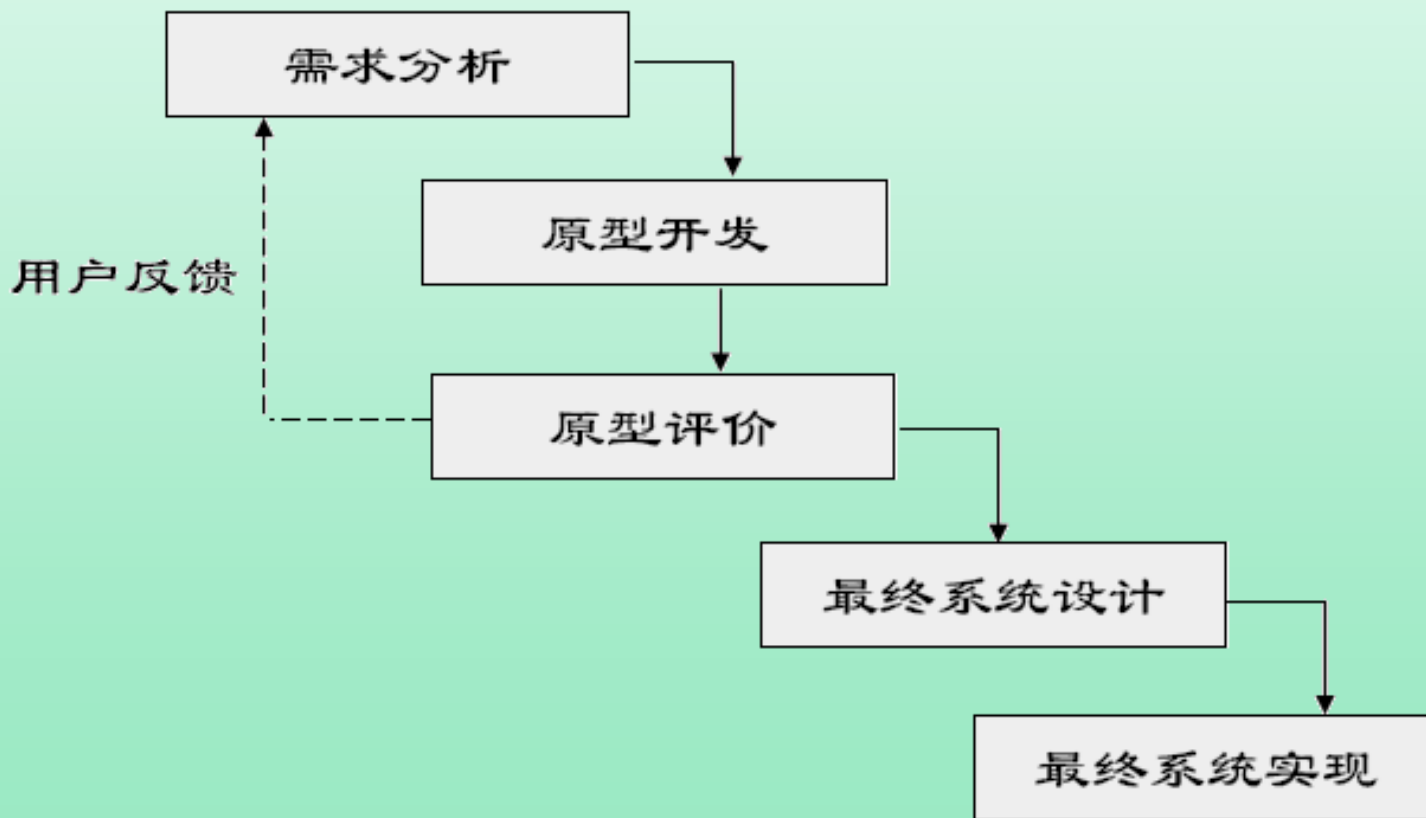
□ 缺点

- 各个阶段的划分完全固定，阶段之间产生大量的文档，极大地增加了工作量
- 开发过程中很难响应客户的变更要求
- 早期的错误可能要等到开发后期的测试阶段才能发现，进而带来严重的后果



快速原型模型

- 快速原型需要迅速建造一个可以运行的软件原型，以便理解和澄清问题，使开发人员与用户达成共识。





快速原型模型

- 目的
 - 减少开发风险和需求不确定性
- 缺点
 - 原型系统的内部结构可能不好
 - 开发人员需要掌握建立快速原型的开发技术和工具
- 适用
 - 小型或中等规模的交互式系统
 - 大型系统的某些部分，例如用户界面
 - 生命周期较短的系统



快速原形法

■ 原型法过程

● 快速分析

系统结构、逻辑结构、用户特征、应用约束、项目管理、项目环境

● 构造原形、修正和改进

● 判定原形完成

● 判定细部的说明

● 整理原形和提供文档

■ 原型法软件开发的重要条件

■ 部分或全部采用原型法



思考题:

- 原形化方法是用户和设计者之间执行的一种交互过程，适用于（ A ）系统。它从用户界面设计开始，首先形成（ B ），用户（ C ）并就（ D ）提出意见。它是一种（ E ）型的设计过程。

A:1. 需求不确定性高的 2. 需求确定的 3. 管理信息 4. 决策支持

B:1. 用户界面使用手册 2. 界面需求分析说明书 3. 系统界面原形 4. 完善的用户界面

C:1. 改进界面的设计 2. 阅读文档资料 3. 模拟界面的运行 4. 运行界面原形

D:1. 同意什么和不同意什么 2. 使用和不使用哪种编程语言 3. 程序的结构 4. 执行速度是否满足要求

E:1. 自外向内 2. 自顶向下 3. 自内向外 4. 自底向上



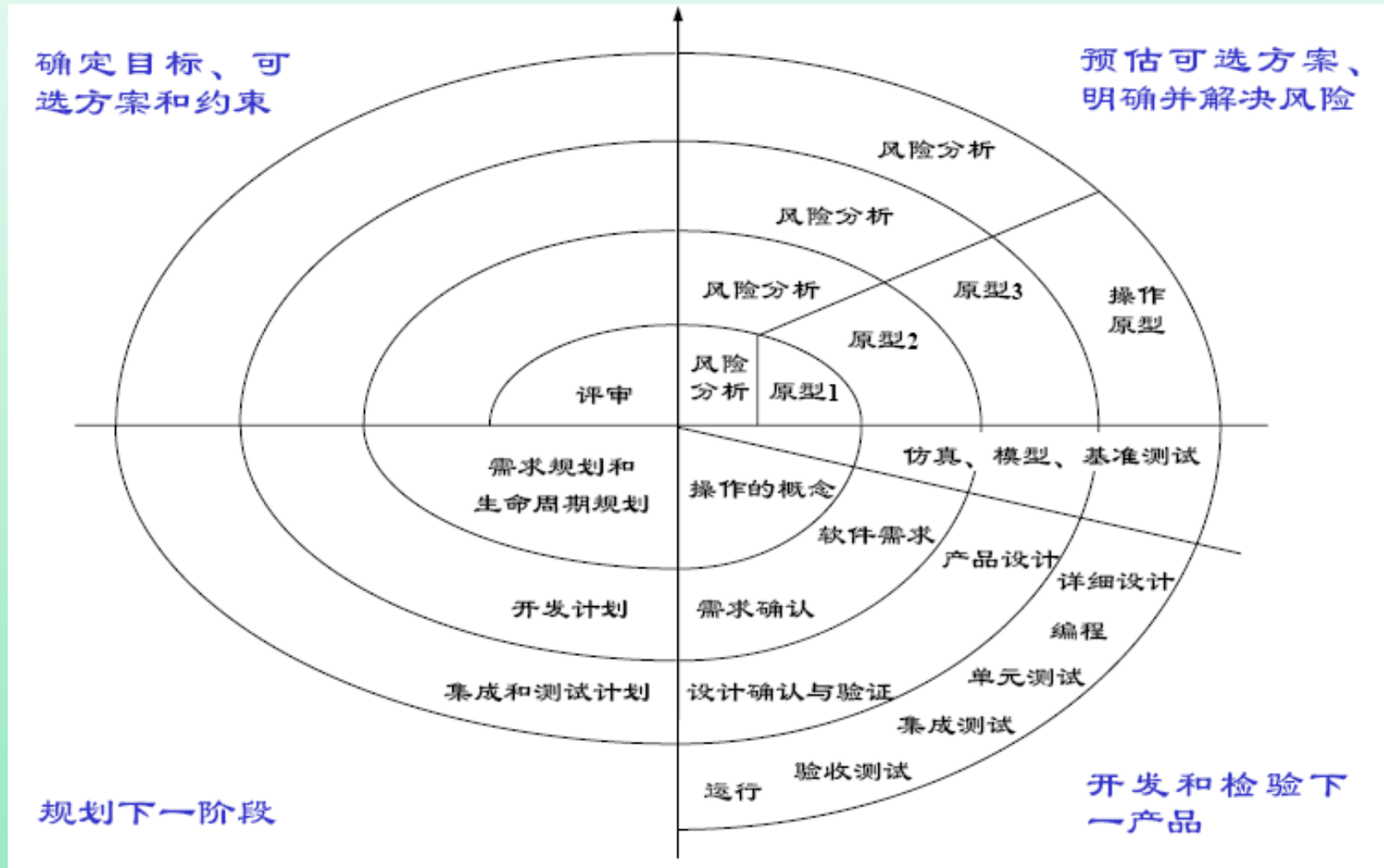


螺旋模型

- 1988年，Barry Boehm提出“螺旋模型”，将瀑布模型和快速原型模型结合，强调风险分析，特别适合于大型复杂的系统。
- 采用周期性的方法，具有众多的中间版本。
- 在早期就能够为客户实证某些概念，进化的开发方式为中心，是快速原型法；在每个项目阶段使用瀑布模型，包括需求定义、风险分析、工程实现和评审等，每次迭代前进一个层次。
- 每一个阶段前引入一个风险识别、风险分析和风险控制，每次循环都标识一个或多个主要风险，直到所有的主要风险因素都被确定。



螺旋模型





螺旋模型

■ 螺旋回线

- 每一个回线表示开发过程的一个阶段
- 例如最中心的第一个回线可能与系统可行性有关，接着第二个回线与需求定义有关，第三个回线与软件设计有关等

■ 四个步骤

- 确定该阶段目标、完成这些目标的可选方案及其约束条件
- 从风险角度分析方案的开发策略，努力排除各种潜在的风险，在需求不适当的情况下可能需要建造原型系统
- 软件开发和验证工作
- 评价该阶段的结果，并规划下一个开发阶段



螺旋模型

- 优点
 - 关注软件的重用
 - 关注早期错误的消除
 - 将质量目标放在首位
 - 将开发阶段与维护阶段结合在一起
- 缺点
 - 契约开发通常需要事先指定过程模型和发布产品
 - 需要风险评估的经验



螺旋模型

经常遇到的问题	螺旋模型的解决方案
用户需求不够充分	允许并鼓励用户反馈信息
沟通不明	在项目早期就消除严重的曲解
刚性的体系 (Overwhelming architectures)	开发首先关注重要的业务和问题
主观臆断	通过测试和质量保证, 作出客观的评估
潜在的不一致	在项目早期就发现不一致问题
糟糕的测试和质量保证	从第一次迭代就开始测试
采用瀑布法开发	在早期就找出并关注风险

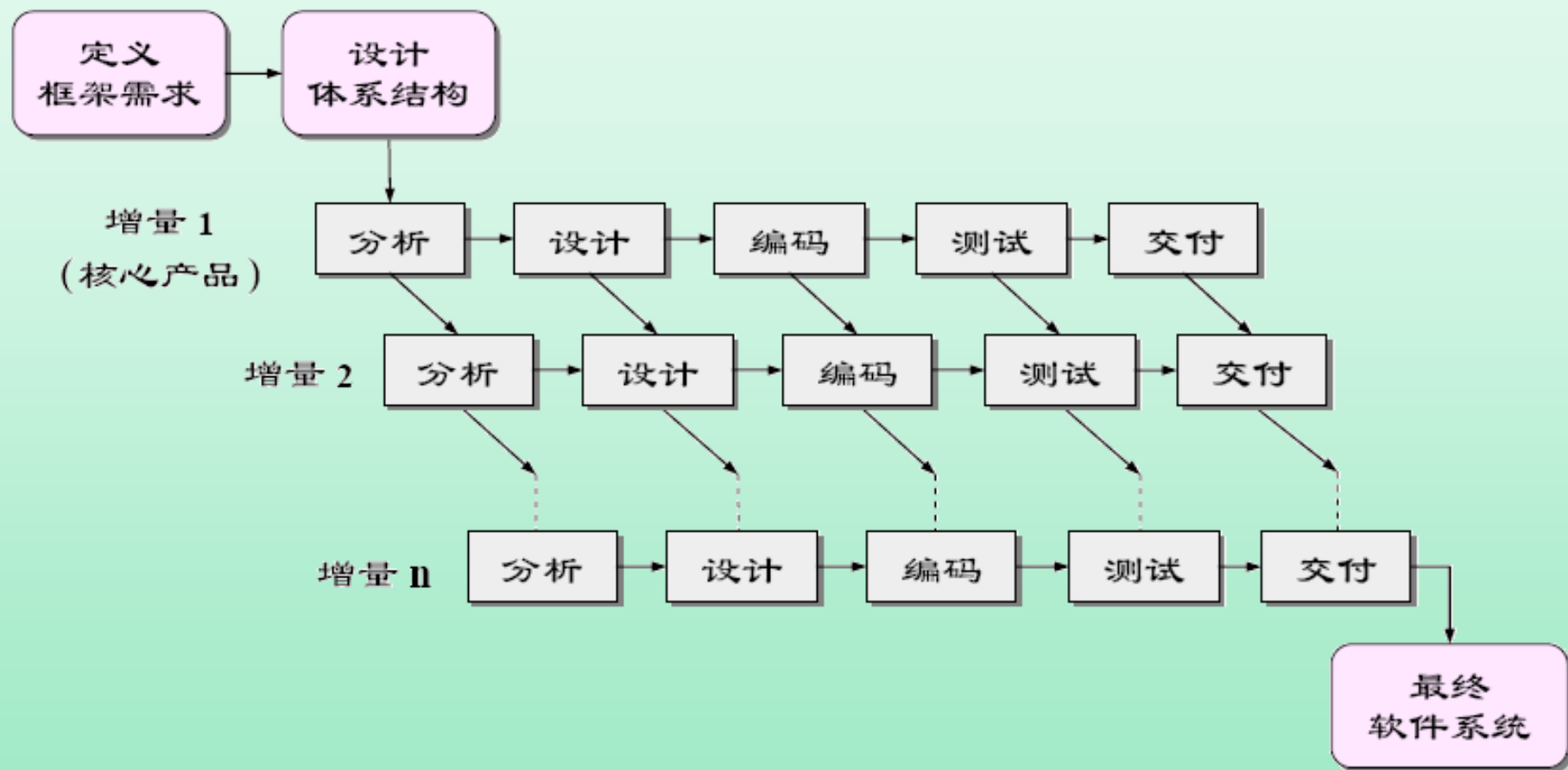


增量模型

- 增量模型融合了瀑布模型的基本成分（重复应用）和原型实现的迭代特征，该模型采用随着日程时间的进展而交错的线性序列，每一个线性序列产生软件的一个可发布的“增量”。
- 增量模型的第1个增量是核心的产品，实现了基本的需求，客户对每一个增量评估，作为下一个增量的新功能，这个过程不断重复，直到产生了最终的完善产品。



增量模型





增量模型

■ 优点

- 产品被分解成若干个构件逐步交付，可以不断地看到可运行中间版本
- 将早期增量作为原型有助于明确后期增量的需求
- 降低开发风险
- 重要功能先交付，从而使其得到最多的测试

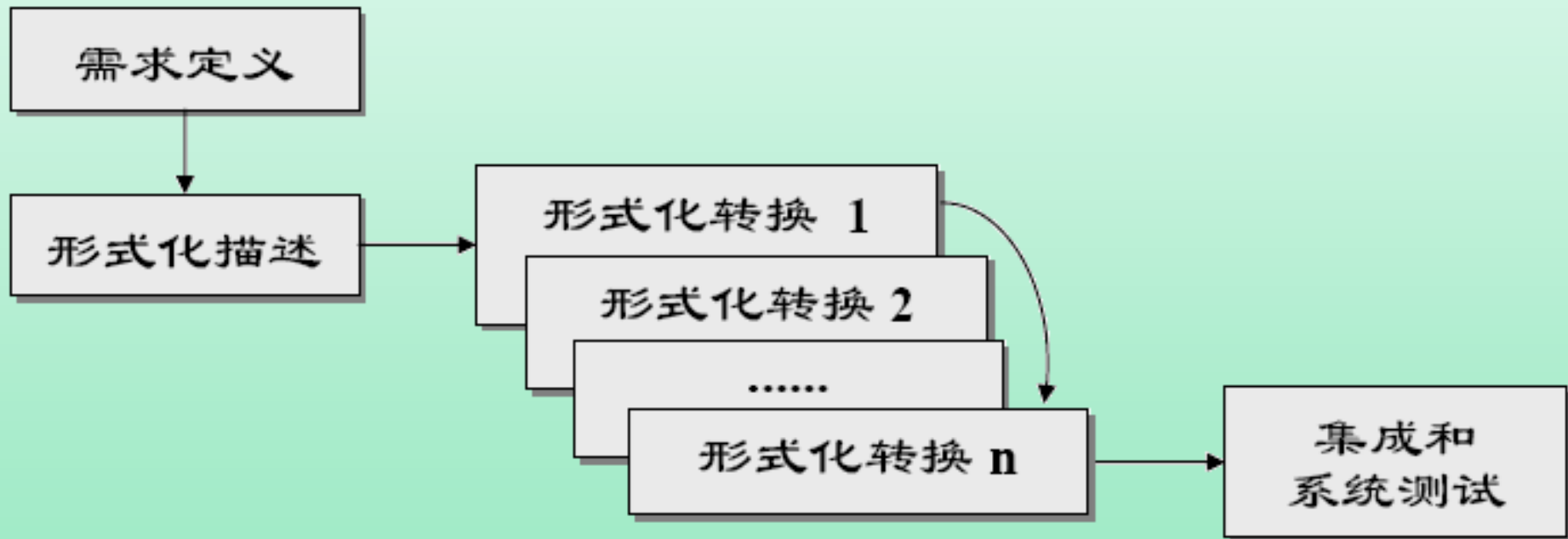
■ 缺点

- 需要软件具备开放式的体系结构
- 需求难以在增量实现之前详细定义，映射和集成比较困难
- 容易退化为边做边改方式，使软件过程控制失去整体性



形式化方法模型

- 形式化方法模型是采用形式化的数学方法将系统描述转换成可执行的程序。





形式化方法模型

■ 适用

- 特别适合于那些对安全性、可靠性和保密性要求极高的软件系统，这些系统需要在投入运行前进行验证

■ 优点

- 由于数学方法具有严密性和准确性，形式化方法开发过程所交付的软件系统具有较少的缺陷和较高的安全性

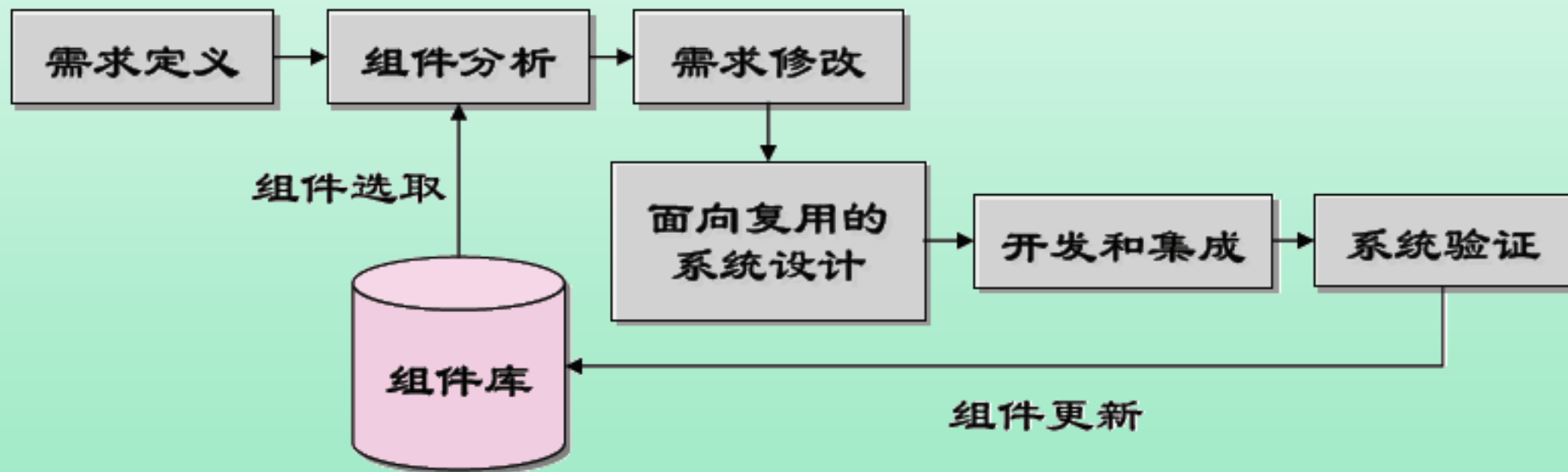
■ 缺点

- 开发人员需要具备一定的技能并经过特殊训练
- 形式化描述和转换是一项费时费力的工作
- 现实应用的系统大多数是交互性强的软件，但是这些系统难以用形式化方法进行描述



基于组件的开发模型

- 基于组件的开发技术是使用可重用的组件或商业组件建立复杂的软件系统。





基于组件的开发模型

- 组件开发技术的两个重要因素
 - 基于组件的软件体系结构
 - 基于组件的开发过程
- 优点
 - 充分体现软件复用的思想
 - 实现快速交付软件
- 缺点
 - 商业组件的修改受到限制，影响系统的演化

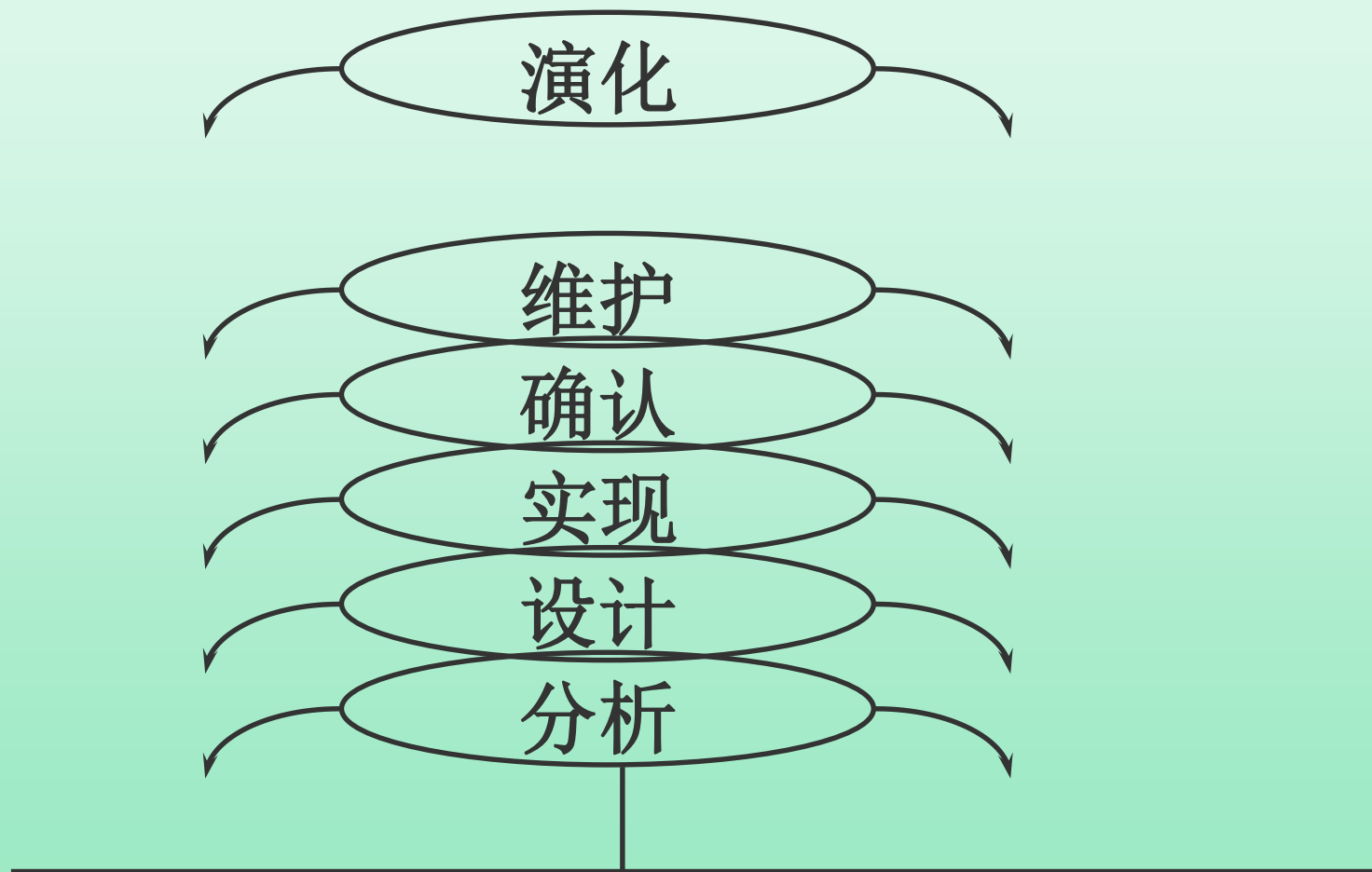


喷泉模型

- 喷泉模型是一种以用户需求为动力，以对象为驱动的模式，主要用于描述面向对象的软件开发过程。该模型认为软件开发过程自下而上周期的各阶段是相互重叠和多次反复的，就像水喷上去又可以落下来，类似一个喷泉。各个开发阶段没有特定的次序要求，并且可以交互进行，可以在某个开发阶段中随时补充其他任何开发阶段中的遗漏。



喷泉模型





喷泉模型

- 喷泉模型主要用于面向对象的软件项目，软件的某个部分通常被重复多次，相关对象在每次迭代中随之加入渐进的软件成分。各活动之间无明显边界，例如设计和实现之间没有明显的边界，这也称为“喷泉模型的无间隙性”。由于对象概念的引入，表达分析、设计及实现等活动只用对象类和关系，从而可以较容易地实现活动的迭代和无间隙。



喷泉模型

■ 喷泉模型的优点

与线性方法不同，该模型的各个阶段没有明显的界限，可以同步进行，提高软件项目开发效率，适应于面向对象的软件开发过程。

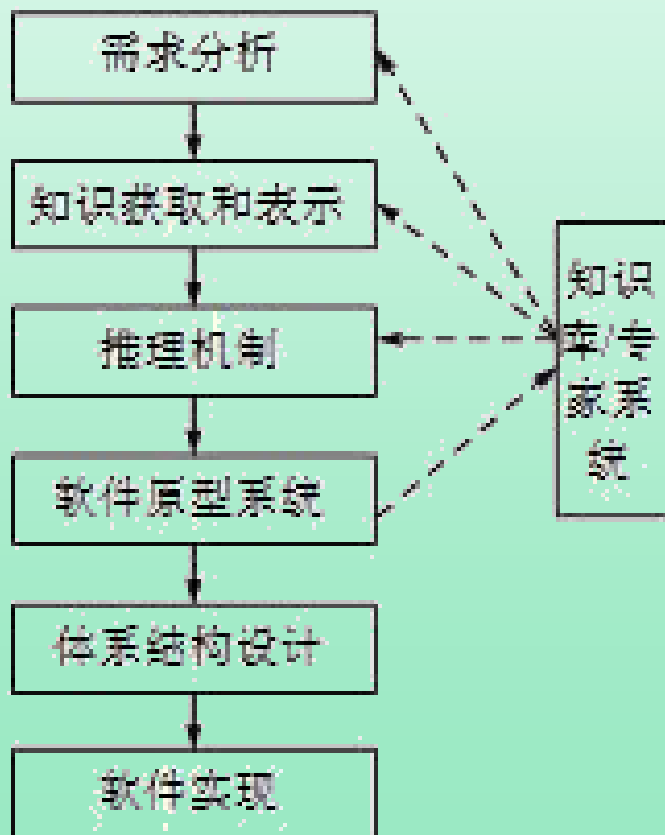
■ 喷泉模型的缺点

由于喷泉模型在各个开发阶段是重叠的，在开发过程中需要大量的开发人员，不利于项目的管理。这种模型要求严格管理文档，使得审核的难度加大。



智能模型

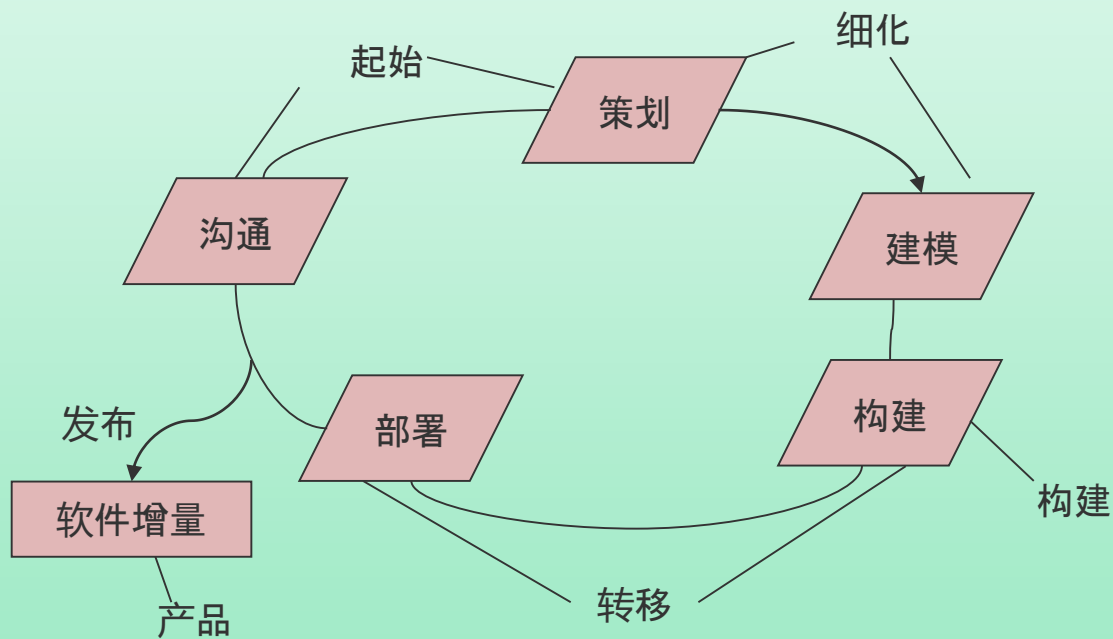
- 基于知识的软件开发模型，综合上述模型，结合专家系统，基于规则采用规约和推理机制，软件维护在规格说明一级进行。





统一过程

一种结合传统方法和敏捷方法的过程



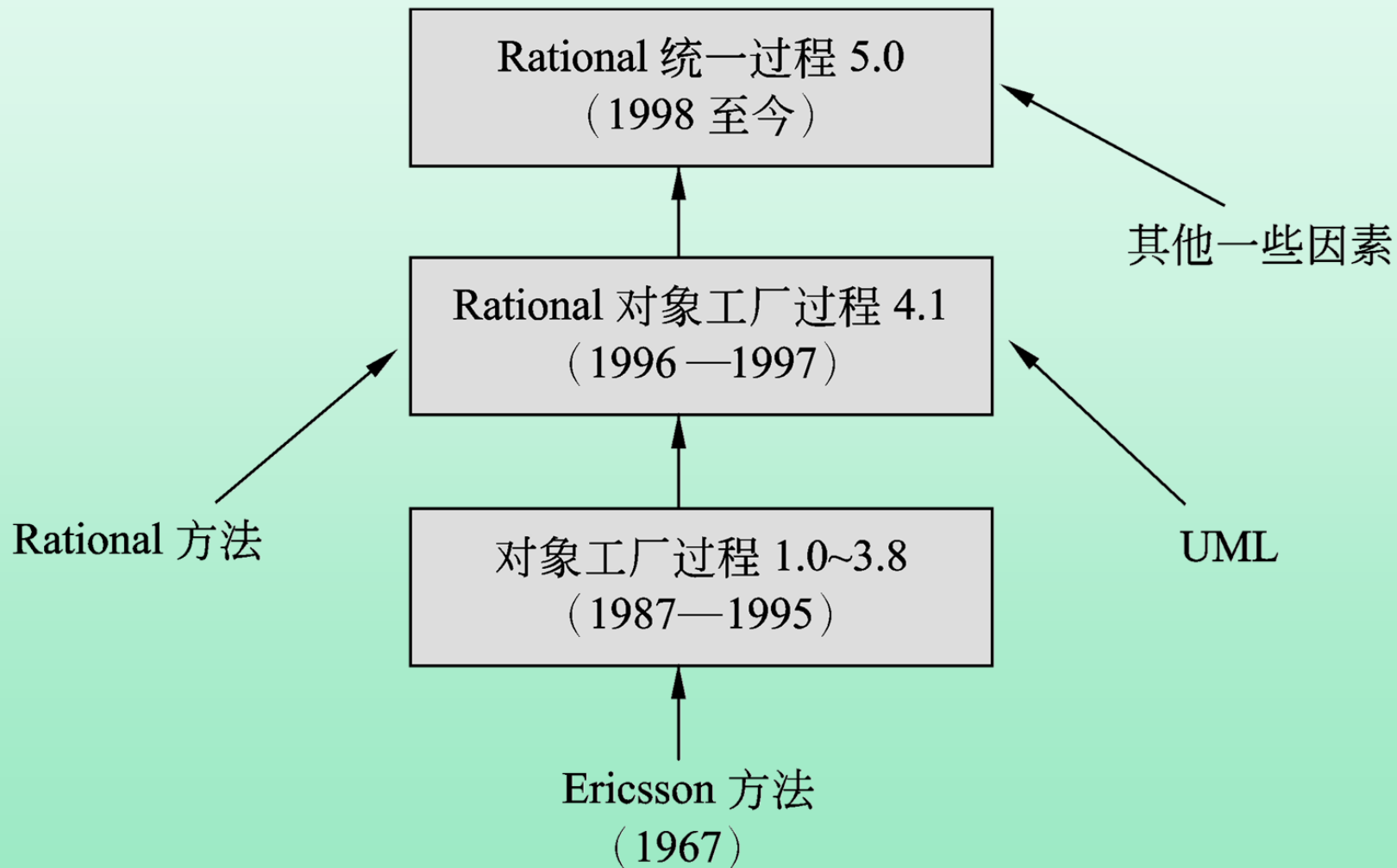


Rational Unified Process

- 软件工程化过程
- Rational公司开发和维护的过程产品
- 提高了团队生产力
- 活动创建和维护模型
- 有效使用UML的指南
- 能对大部分开发过程提供半自动化的工具支持
- 是可配置的过程
- 以适合大范围的项目和机构



Rational 发展过程



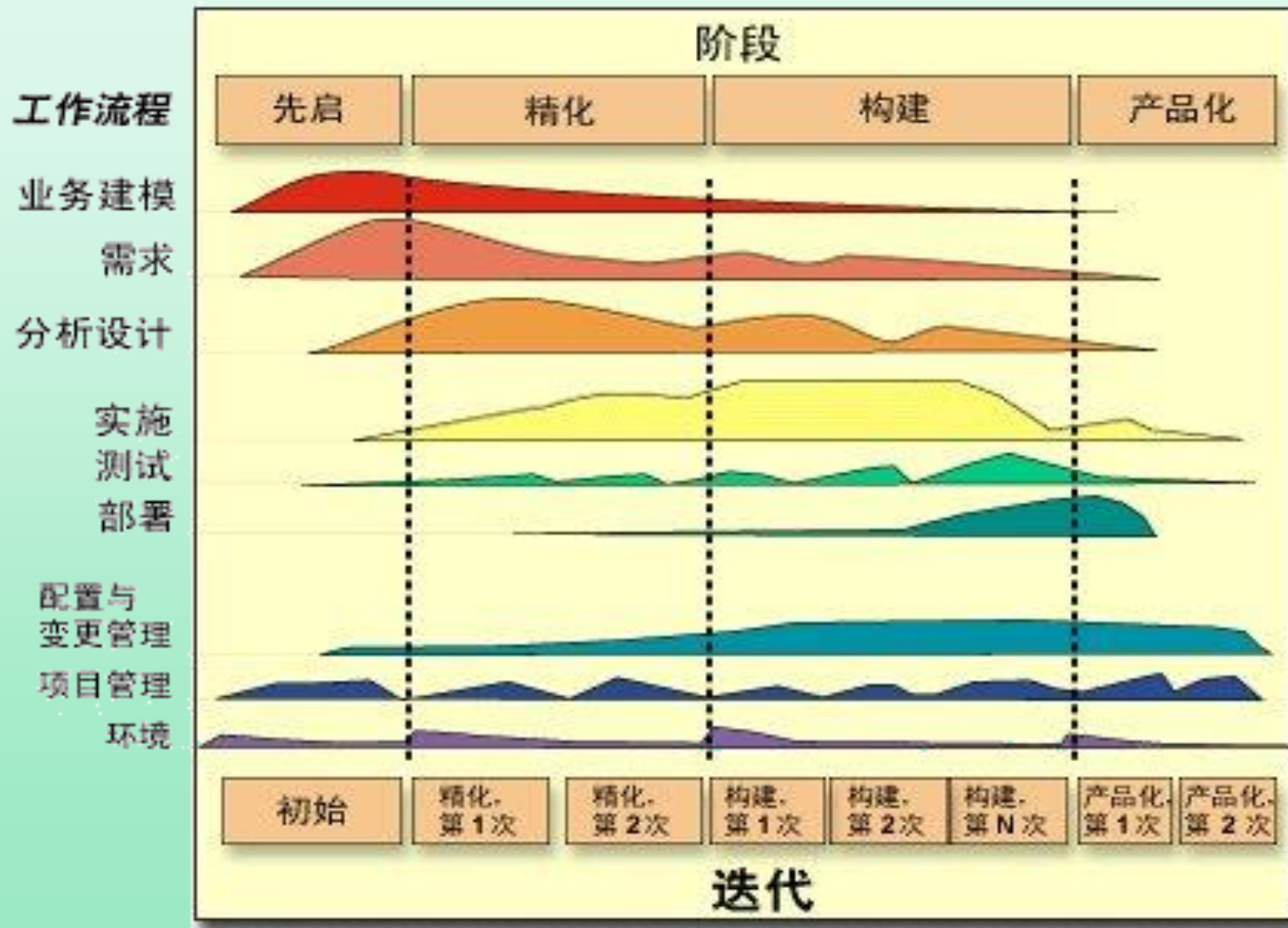


Rational统一过程

- ❑ 迭代式开发
- ❑ 需求管理
- ❑ 使用基于构件的体系结构
- ❑ 可视化建模
- ❑ 验证软件质量
- ❑ 控制软件变更

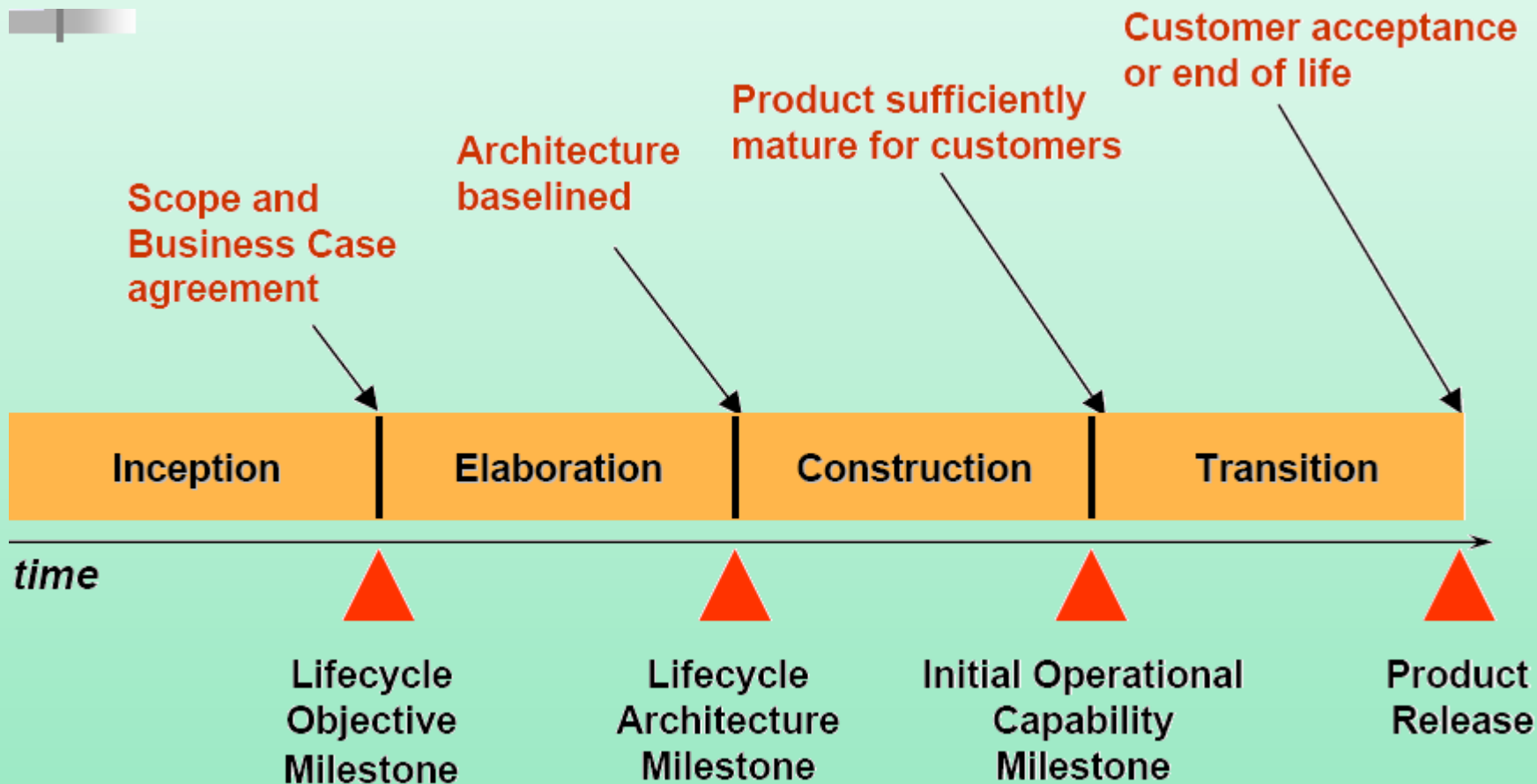


统一过程





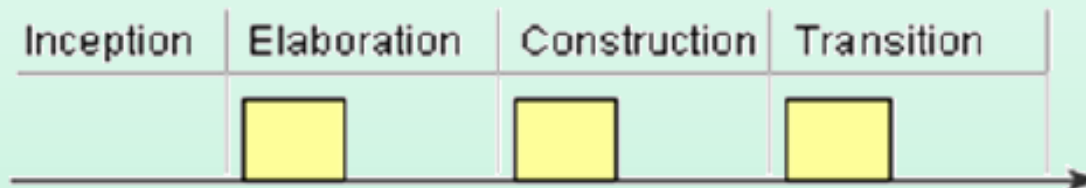
统一过程



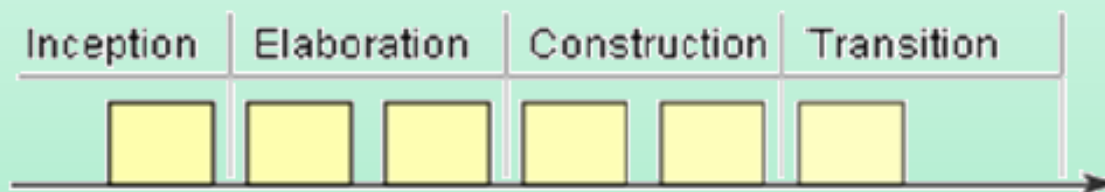


统一过程

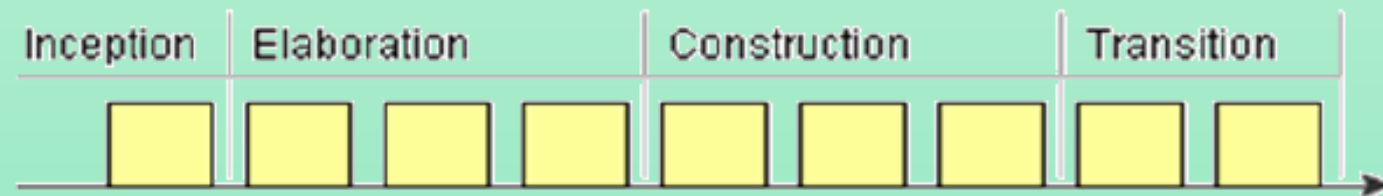
Low-End



Medium



High-End





统一过程

- 起始阶段：愿景文档、初始用例模型、初始项目术语表、初始业务用语、初始风险评估、项目计划、业务模型、原型
- 细化阶段：用例模型、补充需求、分析模型软件体系结构描述、可执行的体系结构原型、初步设计模型、修订的风险列表、项目计划初始用户手册
- 构建阶段：设计模型、软件构件、集成的软件增量、测试计划及步骤、测试用例、支持文档
- 转化阶段：提交的软件增量、Beta测试报告、综合用户反馈



Rational | 统一过程方法与工具

■ 方法

- 用例及用例驱动
- 以架构为中心
- 采用UML可视化建模
- 面向对象设计与实现

■ 工具

- Rational Rose

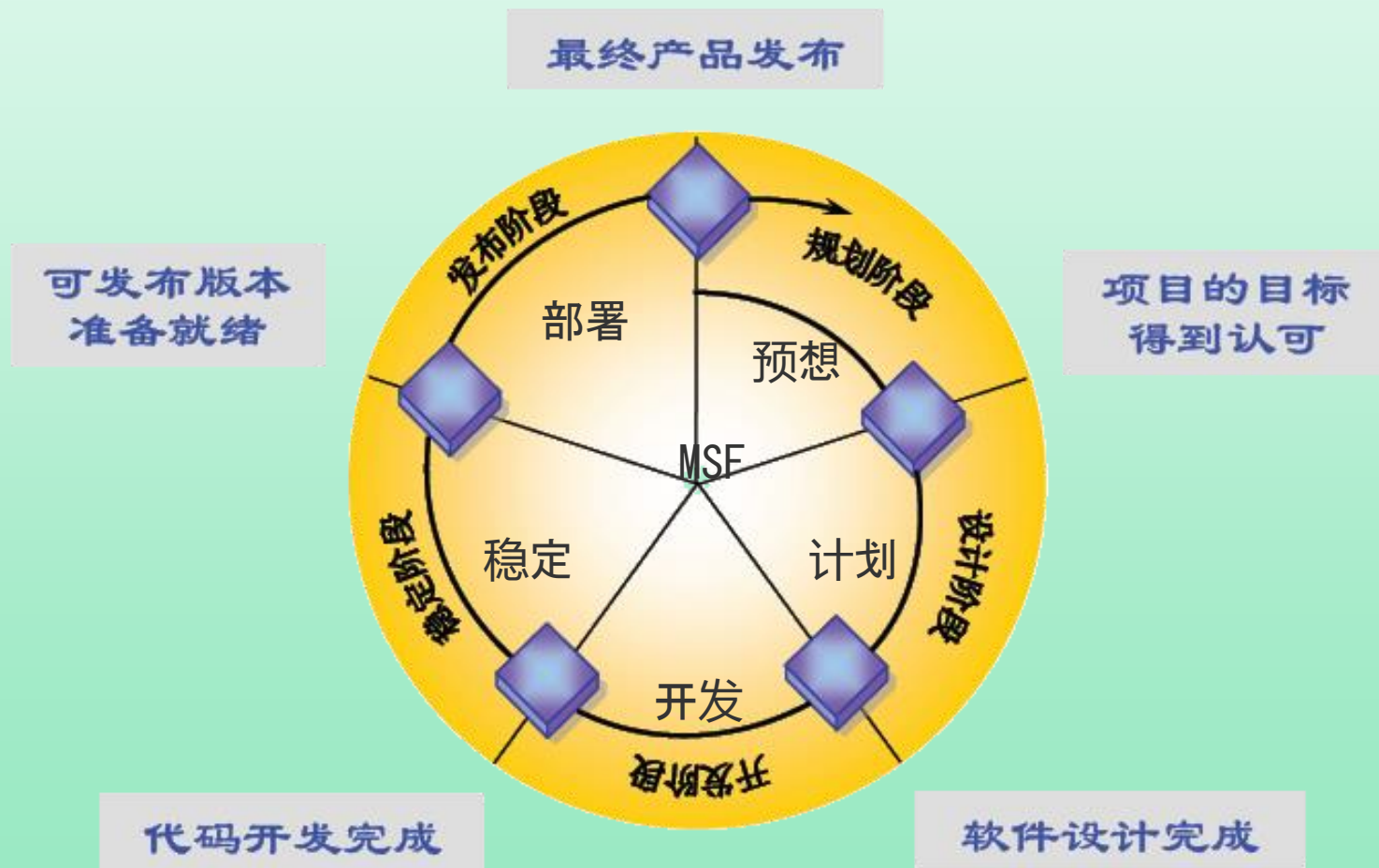


微软公司过程MSF

- 微软公司的开发管理原则
 - 以目标驱动的开发过程
 - 具有外部可见的里程碑
 - 基于多版本的产品发布
 - 并行协作的小型化团队
 - 经常性的同步和稳定



微软公司的软件开发过程MSF





微软公司的软件开发过程

□ 规划阶段

- 开展市场调查研究，结合公司战略形成产品的远景目标。
- 愿境/范围说明书、风险评估说明书、项目组织结构说明书

□ 设计阶段

- 根据产品的远景目标，完成软件的功能规格说明和总体设计，确定开发的主要进度。
- 目标和功能说明书、风险管理计划、总体计划（进度）表、规范



微软公司的软件开发过程

❑ 开发阶段

- ❑ 整个开发任务划分成若干个递进的阶段，并设置成M1、M2、……、Mn等内部里程碑，在每个里程碑都提交阶段性的工作成果。
- ❑ 源代码、配置信息、功能说明书、使用支持、测试说明书（用例）

❑ 稳定阶段

- ❑ 实行全面的内部和外部测试，最终形成可发布的RTM版本。
- ❑ 版本、注释、使用支持、测试结果、源代码、项目文档、评审记录

❑ 发布阶段

- ❑ 在确认产品质量符合发布标准之后，发布产品及其相关的消息。
- ❑ 运营和支持IS、程序和过程、报告和日志、文档、总结、使用报告、下步计划



微软公司的软件开发过程

■ *M1* (1/3功能)

- 开发（设计、编码）
- 可用性实验 - 内部发布测试
- 系统构建
- 程序调试
- 集成
- 代码稳定
- 缓冲时间

■ *M2* (1/3功能)

- 开发
- 可用性实验
- 内部发布测试
- 系统构建
- 程序调试
- 集成
- 代码稳定
- 缓冲时间

● *M3* (1/3功能)

- 开发
- 可用性实验
- 内部发布测试
- 系统构建
- 程序调试
- 集成
- 代码稳定
- 缓冲时间
- “零缺陷”发布
- 最终产品发布



微软公司的软件开发过程

■ 递进式的开发策略

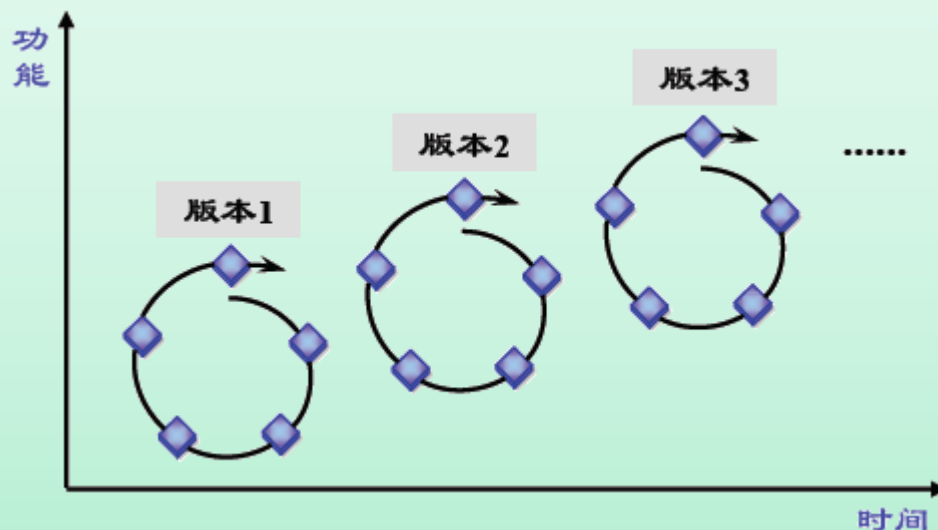
- 解决问题的及时性
- 不确定和变更因素的可控性
- 缩短产品上市周期

■ 缓冲时间

- 在每一个递进阶段，缓冲时间有利于开发人员应对变更、开发难题和时间延迟等问题

■ 并行与同步机制

- 小规模团队并行开发、每天的系统构建





实例：微软Window2000项目

- 人员5000
- 分四组：程序管理员、程序开发人员、测试员和合作伙伴，客户
- 时间：下午3：00~8：00构建产品，8：00~第二天9：00测试，9：00由程序管理人员写下测试报告以供下午开发人员参考。



五、信息系统工程方法

- 需求工程
- 系统建模
- 分析与设计方法



基于UML的过程建模

- 用户模型视图：从用户的视角来表示系统。用例（Use-case）描述使用场景，可用于用户模型视图的建模方案。
- 结构模型视图：从系统内部来分析数据和功能，属于静态结构建模。
- 行为模型视图：描述系统动态或行为方面的各种元素间交互或协作关系，属于动态结构建模。
- 实现模型视图：针对如何构建（实现）系统的结构和行为时的表示。
- 环境模型视图：表示待实现的系统环境的结构和行为。



UML图

- 用例模型：用例图、序列图、协作图、状态图和活动图
- 分析模型：类图和对象图（包括子系统和包）、序列图、协作图、状态图和活动图。
- 设计模型：类图和对象图（包括子系统和包）、序列图、协作图、状态图和活动图。
- 开发模型：配置图（包括活动类和组件）、序列图、协作图。
- 实现模型：组件图、序列图和协作图。
- 测试模型：测试模型引用了所有其它模型，所以使用所对应的所有视图。



基于IDEF3的过程建模

- 美国空军集成计算机辅助制造（ICAM），包含3种方法：功能建模（IDEF0）、信息建模（IDEF1）和动态建模（IDEF2）。
- 随后增加了不少IDEF方法，如数据建模扩展版本（IDEF1X）、[过程描述获取方法（IDEF3）](#)、面向对象的设计方法（IDEF4）、实体论（Ontology）描述获取方法（IDEF5）、设计理论（rationale）获取方法（IDEF6）、人机交互设计方法（IDEF8）、业务约束发现方法（IDEF9）、网络设计方法（IDEF14）等。



complementary



中国人民大学信息学院徽章



标志释义： 标识以信字为创意点进行图形创作，左侧取材于人大校标，右侧以英文info 竖排并巧妙变形为“言”字与左侧篆体“人”字自然的结合成为一个“信”，在视觉上既表达了人民大学的整体又反映出信息学院的特点。标识信息感强烈而又有独特内涵，INFO在表现篆体“言”的同时还有目的图形变化，使之具有集成电路的抽象形式和信息流通传递和互通的抽象概念。